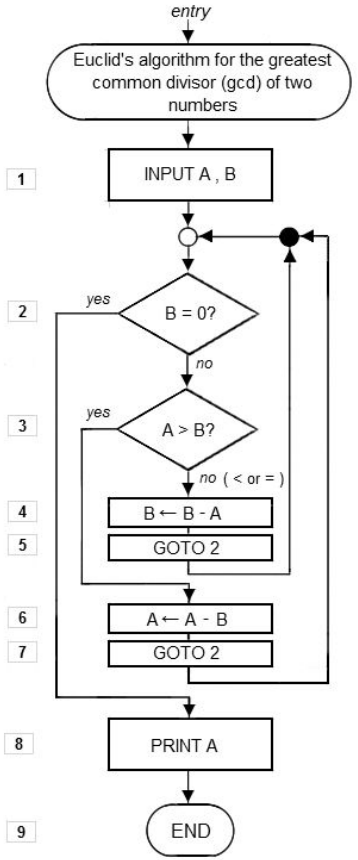
Algorithmie, C’est quoi un algorithme ?

L'**algorithmique** est l'étude et la production de règles et techniques qui sont impliquées dans la définition et la conception d'algorithmes, c'est-à-dire de processus systématiques de résolution d'un problème permettant de décrire précisément des étapes pour résoudre un problème algorithmique.

Source :
https://fr.wikipedia.org/wiki/Algorithmique





L'**algorithmique** est la capacité d’une personne à découper un problème complexe en sous parties plus faciles à comprendre afin de faire exécuter une tâche à une machine ou une personne.

Source : MOI

Des algorithmes que nous utilisons tous les jours :

- La liste des courses
- Une recette de cuisine
- Suivre un itinéraire
- Les étapes pour monter un meuble
- …

Chaque étape d'un algorithme doit être **CLAIREMENT EXPLICITE** pour ne pas faire planter le code (ou la personne) et faire en sorte que l'algorithme s'exécute correctement.

La différence entre un humain et une machine est que cette dernière n’est pas capable de comprendre un sous-entendu et va faire bêtement ce qu’il y a d’écrit. Moins on est précis, plus on laisse la machine interpréter ce qu’elle comprend, moins l’algorithme s’exécute comme on veut.



Source : https://www.youtube.com/watch?v=FN2RM-CHkuI



https://www.youtube.com/watch?v=tbmKIErjnns

Le JavaScript, C’est quoi ?

Algorithmie - Le JavaScript, c’est quoi ?

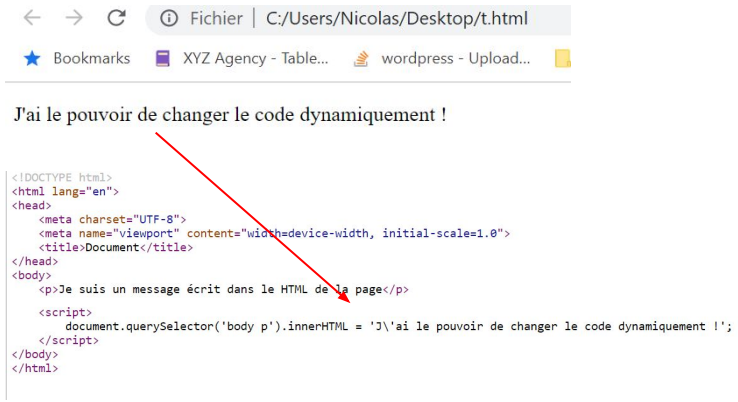
Le JavaScript est un langage de programmation créé en 1995. Le JavaScript est aujourd’hui l’un des langages de programmation les plus populaires et il fait partie des langages web dits « standards » avec le HTML et le CSS. Son évolution est gérée par le groupe ECMA International qui se charge de publier les standards de ce langage.

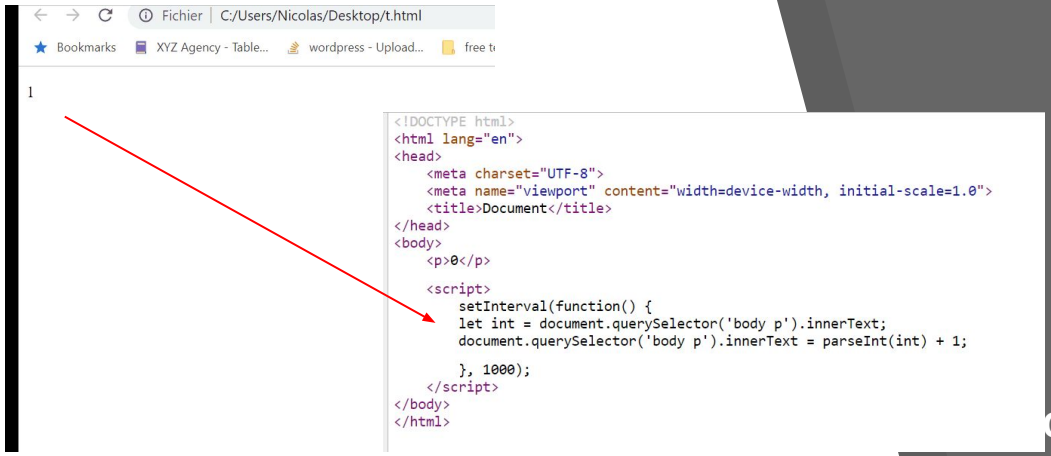
- Le JavaScript est un langage dynamique ;
- Le JavaScript est un langage (principalement) côté client ;
- Le JavaScript est un langage interprété ;
- Le JavaScript est un langage orienté objet.

Algorithmie - Le JavaScript, c’est quoi ?

Le JavaScript est un langage dynamique :

Le JavaScript peut modifier le contenu d'une page web (modifier, ajouter, supprimer un texte d'une page par exemple) , faire des calculs (1*3 par exemple), aller chercher des données sur d'autres sources (via des API), structurer des données (sous forme de tableau ou objet), stocker des informations via des cookies, ...On oppose généralement les langages « dynamiques » aux langages « statiques » comme le HTML et le CSS.





Algorithmie - Le JavaScript, c’est quoi ?

Le JavaScript est un langage (principalement) côté client :

C'est une code "front-end", c'est à dire que les navigateurs web peuvent l'interpréter.
Cependant, il peut également être comme langage serveur (nodeJS).
D’ailleurs, si vous voyez le code dans la source, c’est que c’est un code client, un code serveur est invisible depuis le navigateur.

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
</head>
<body>
    <p>0</p>

    <script>
        setInterval(function() {
        let int = document.querySelector('body p').innerText;
        document.querySelector('body p').innerText = parseInt(int) + 1;

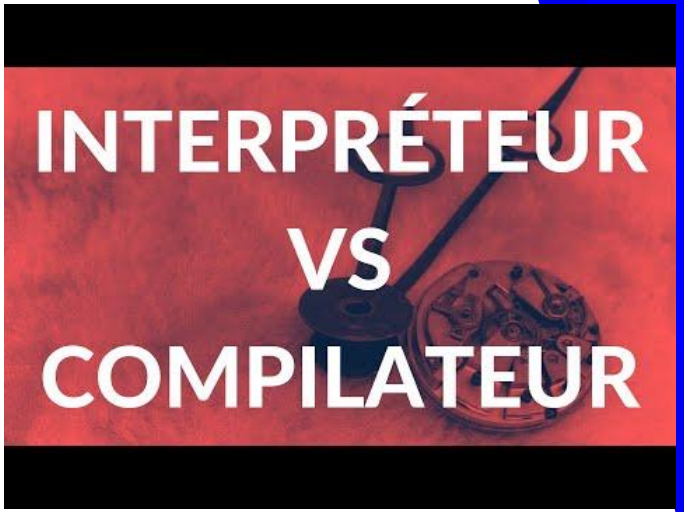
        }, 1000);
    </script>
</body>
</html>
```

Algorithmie - Le JavaScript, c’est quoi ?

Le JavaScript est un langage interprété :

Le JavaScript est un langage interprété. Cela signifie qu’il va pouvoir être exécuté directement sous réserve qu’on possède le logiciel interpréteur. C'est à dire un navigateur web. C'est lui qui va "traduire" notre code pour lui faire faire ce que l'on veut.

L'opposé du langage interprété est le langage compilé (comme le SCSS ou le C par exemple). https://www.youtube.com/watch?v=4lXp_89c3RU



Algorithmie - Le JavaScript, c’est quoi ?

Le JavaScript est un langage orienté objet :

Trop tôt pour en parler mais, pour info, l'objet du JavaScript s'appelle le JSON (JavaScript Object Notation) et qu'il sert à structurer des informations. Un peu comme les tableaux. On va dire que la différence entre un tableau et un objet est que l’objet peut stocker des fonctions. On y reviendra...

```
    <script>
        var personne = {
          nom: ['Jean', 'Martin'],
          age: 32,
          sexe: 'masculin',
          interets: ['musique', 'skier'],
          bio: function() {
           alert(this.nom[0] + ' ' + this.nom[1] + ' a ' + this.age + ' ans. Il aime ' +
               this.interets[0] + ' et ' + this.interets[1] + '.');
          },
          salutation: function() {
            alert('Bonjour ! Je suis ' + this.nom[0] + '.');
          }
        };


    </script>
</body>
```

Le JavaScript n’est pas du Java !!!

Ce sont 2 codes complètement différents !

Trop de personnes confondent ces 2 codes mais plus vous.

Si vous trouvez que JavaScript est trop long à dire, dites JS.





https://www.youtube.com/watch?v=cdDPQkF7hRA

Algorithmie - Le JavaScript, c’est quoi ?

Le JS s’écrit soit entre 2 balises script OU directement dans un fichier avec une extension en .js (c’est plus propre)

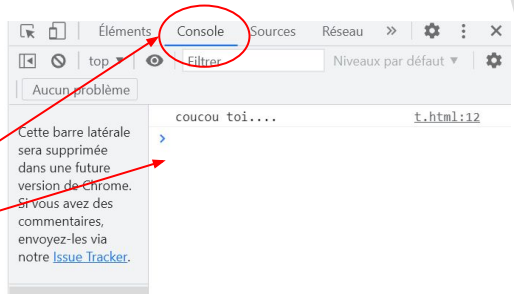
```
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
</head>
<body>
    <p>Lorem ipsum dolor sit amet consectetur adipisicing elit. Facilis esse, incidunt culpa, omnis
    ipsum voluptatibus nostrum maiores suscipit veritatis, ipsam aliquam nesciunt voluptates molestias
    reiciendis illum nobis. Molestiae, odio voluptates.</p>

    <script>
        var personne = {
          nom: ['Jean', 'Martin'],
          age: 32,
          sexe: 'masculin',
          interets: ['musique', 'skier'],
          bio: function() {
           alert(this.nom[0] + ' ' + this.nom[1] + ' a ' + this.age + ' ans. Il aime ' + this.interets[0]
                + ' et ' + this.interets[1] + '.');
          },
          salutation: function() {
            alert('Bonjour ! Je suis ' + this.nom[0] + '.');
          }
        }
```

```
        </div>
    </footer>
    <!-- //footer -->
    <a href="#home" class="scroll" id="toTop" style="display: block;"> <span id="toTopHover" style="opacity: 1;"> </span></a>
    <script type="text/javascript" src="https://www.planete-velo.fr/wp-content/themes/PlanetVelo/public/js/bootstrapp.js"></script>
    <script src="https://www.planete-velo.fr/wp-content/themes/PlanetVelo/public/js/search.js"></script>
    <script type="text/javascript" src="https://www.planete-velo.fr/wp-content/themes/PlanetVelo/public/js/move-top.js"></script>
    <script type="text/javascript" src="https://www.planete-velo.fr/wp-content/themes/PlanetVelo/public/js/jquery.easing.min.js"></script>
    <script src="https://www.planete-velo.fr/wp-content/themes/PlanetVelo/public/myPlugins/allMyPlugins.js"></script>
    <script src="https://www.planete-velo.fr/wp-content/themes/PlanetVelo/public/js/functions.js"></script>
    <!-- js for portfolio lightbox -->
    <script src="https://www.planete-velo.fr/wp-content/themes/PlanetVelo/public/js/jquery.chocolat.js"></script>
    <script src="https://www.planete-velo.fr/wp-content/themes/PlanetVelo/public/js/main.js"></script>
    <script src="https://www.planete-velo.fr/wp-content/themes/PlanetVelo/public/js/Tracker.js"></script>
```

Algorithmie - Le JavaScript, c’est quoi ?

Lorem ipsum dolor sit amet consectetur adipisicing elit. Facilis esse, incidunt culpa, omnis ipsum voluptatibus nostrum maiores suscipit veritatis, ipsam aliquam nesciunt voluptates molestias reiciendis illum nobis. Molestiae, odio voluptates.




```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
</head>
<body>
    <p>Lorem ipsum dolor sit amet consectetur adipisicing elit. Facilis esse, incidunt culpa, omnis
    ipsum voluptatibus nostrum maiores suscipit veritatis, ipsam aliquam nesciunt voluptates molestias
    reiciendis illum nobis. Molestiae, odio voluptates.</p>

    <script>
        console.log('coucou toi....');
    </script>
</body>
</html>
```

Quand on code en JS, on ouvre la console (clique droit > inspecter > onglet “console”) dans le navigateur.

C’est dans cet endroit où on va pouvoir constater les éventuelles Erreurs. De base, le JS est invisible…

Algorithmie, Les Variables

Algorithmie - Les variables

Quelques définitions :

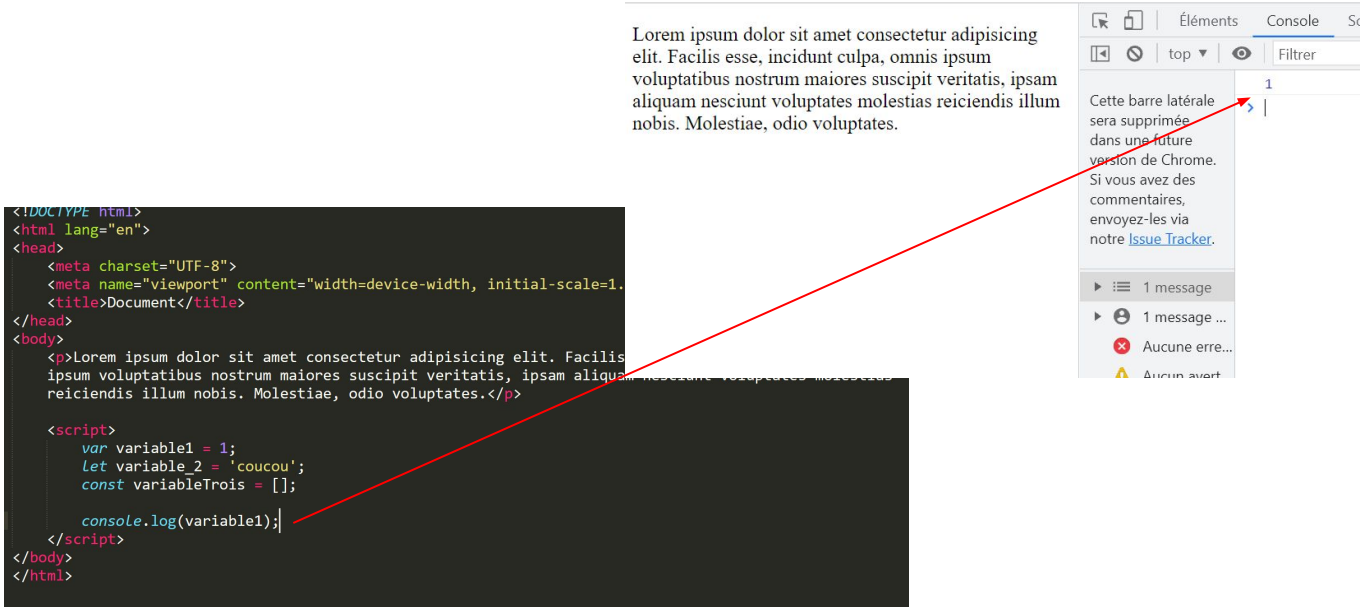
String : chaine de caractère (une phrase en gros, entre guillemet)
Number : un chiffre/nombre entier
Boolean : true /false - 0 / 1 - ON / OFF
Var : initialisation d'une variable qui peut changer (va devenir obsolète)
Const : initialisation d'une variable qui peut ne peut pas changer
Let : initialisation d'une variable qui peut changer
Array : c'est un tableau (indéxé, associatif, multi-dimensionnel)
Concaténation : le fait de mettre une variable dans un string

Algorithmie - Les variables

```
<script>
    var variable1 = 1;
    let variable_2 = 'coucou';
    const variableTrois = [];
</script>
```

Une variable est une boite servant à stocker des informations de manière temporaire, comme une chaine de caractères (variable_2), un nombre (variable1), un tableau (variableTrois) par exemple. Les variables sont l'une des constructions de base du JavaScript et vont être des éléments qu'on va énormément utiliser. Nous allons illustrer leur utilité par la suite.

Algorithmie - Les variables



On voit bien que si je souhaite afficher la variable “variable1” dans la console,
On voit la valeur 1, celle qu’on lui a attribué.
console.log(‘...’) affiche dans la console le contenu entre parenthèses.

Algorithmie - Les variables

```
<script>
    var variable1 = 1;
    let variable_2 = 'coucou';
    const variableTrois = [];
</script>
```

- Le nom d'une variable doit obligatoirement commencer par une lettre ou un underscore (_) et ne doit pas commencer par un chiffre ;

- Le nom d'une variable ne doit contenir que des lettres, des chiffres et des underscores mais pas de caractères spéciaux ;

- Le nom d'une variable ne doit pas contenir d'espace.

Algorithmie - Les variables

```
<script>
    var variable1 = 1;
    let variable_2 = 'coucou';
    const variableTrois = [];
</script>
```

Pourquoi ces petits mots devant les variables ?

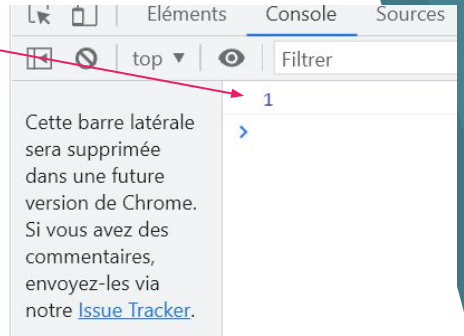
Quand une variable est utilisée pour la première fois, il faut l'initialiser. Et pour ce faire, on utilise soit **var**, **let** ou encore **const** .
Cela permet aussi de mieux organiser ses infos et son code.

Algorithmie - Les variables

```
<script>
    variable1 = 1;
    let variable_2 = 'coucou';
    const variableTrois = [];

    console.log(variable1);
</script>
```

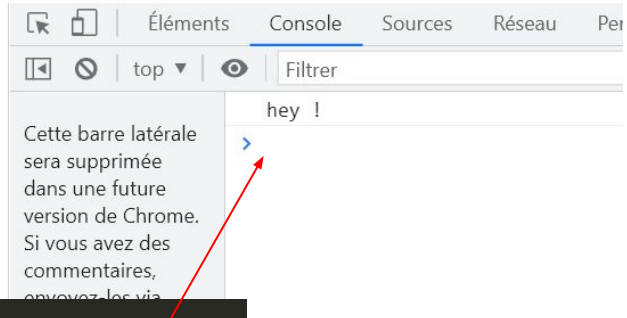
Lorem ipsum dolor sit amet consectetur adipisicing elit. Facilis esse, incidunt culpa, omnis ipsum voluptatibus nostrum maiores suscipit veritatis, ipsam aliquam nesciunt voluptates molestias reiciendis illum nobis. Molestiae, odio voluptates.



Après, on peut voir que JS fonctionne normalement si on ne l'initialise pas correctement. Mais ça se fait pas.

Algorithmie - Les variables

Lorem ipsum dolor sit amet consectetur adipisicing elit. Facilis esse, incidunt culpa, omnis ipsum voluptatibus nostrum maiores suscipit veritatis, ipsam aliquam nesciunt voluptates molestias reiciendis illum nobis. Molestiae, odio voluptates.




```
<script>
    var variable1 = 1;
    variable1 = 'belette';
    variable1 = 'hey !';
    console.log(variable1);
</script>
```

Le code est lu de haut en bas. De ce fait, une variable peut être réécrite (pas toute) et changer de valeur. C'est la dernière valeur qui écrase les autres...

```
<script>
    var variable1 = 1;
    let variable_2 = 'coucou';
    const variableTrois = [];
</script>
```

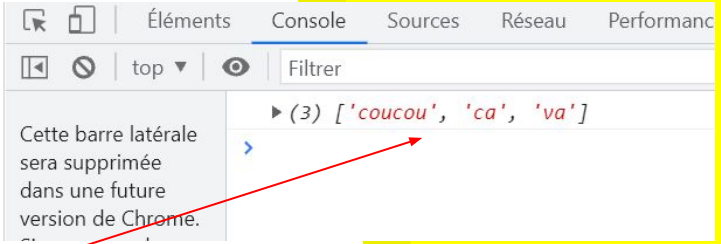
Ok, mais c’est quoi la différence entre var, let et const ????

Il faut savoir que ‘var’ est en passe d’être obsolète. Il ne faut pas l’utiliser. On peut encore la voir dans certains codes. Au début de JS, il n’y avait que ‘var’ pour initialiser les variables.

Algorithmie - Les variables

Lorem ipsum dolor sit amet consectetur adipisicing elit. Facilis esse, incidunt culpa, omnis ipsum voluptatibus nostrum maiores suscipit veritatis, ipsam aliquam nesciunt voluptates molestias reiciendis illum nobis. Molestiae, odio voluptates.

```
<script>
    let variable1;
    variable1 = 20;
    variable1 = ['coucou','ca','va'];
    console.log(variable1);
</script>
```

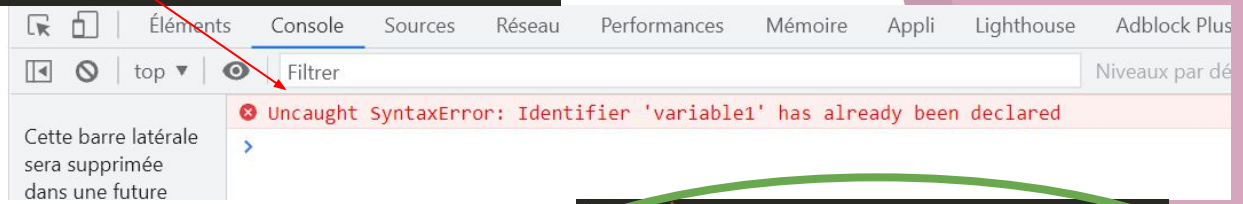


“Let” permet de pouvoir réécrire les variables à l’infini (comme “var”)

Algorithmie - Les variables

```
<script>
    let variable1;
    let variable1 = 20;
    let variable1 = ['coucou','ca','va'];
    console.log(variable1);
</script>
```

Lorem ipsum dolor sit amet consectetur adipisicing elit. Facilis esse, incidunt culpa, omnis ipsum voluptatibus nostrum maiores suscipit veritatis, ipsam aliquam nesciunt voluptates molestias reiciendis illum nobis. Molestiae, odio voluptates.

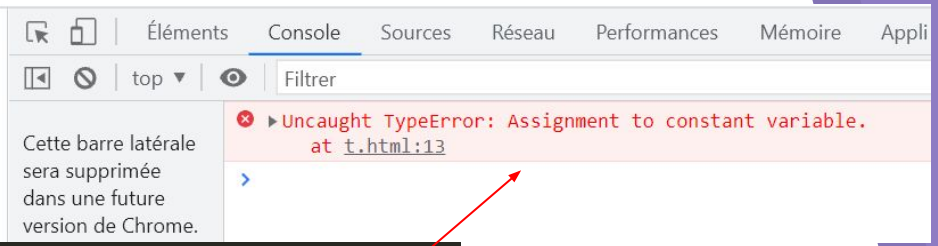



```
<script>
    let variable1;
    variable1 = 20;
    variable1 = ['coucou','ca','va'];
    console.log(variable1);
</script>
```

“Let” ne peut être déclarée qu’une seule fois (pas comme “var”)

Algorithmie - Les variables

Lorem ipsum dolor sit amet consectetur adipisicing elit. Facilis esse, incidunt culpa, omnis ipsum voluptatibus nostrum maiores suscipit veritatis, ipsam aliquam nesciunt voluptates molestias reiciendis illum nobis. Molestiae, odio voluptates.

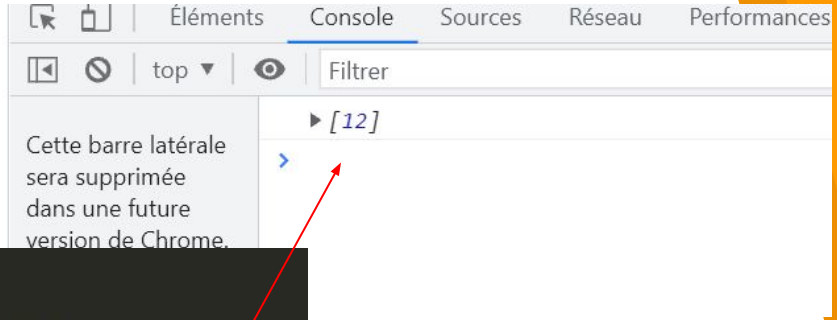



```
<script>
    const variable = 12;
    variable = 13;
    console.log(variable);
</script>
</body>
```

"Const" (qui veut dire "constante") ne peut pas être réassignée (modifiée).
Elle garde la valeur qu'elle a reçu lors de son initialisation.
En gros, c'est une variable qui ne change jamais.

Algorithmie - Les variables

Lorem ipsum dolor sit amet consectetur adipisicing elit. Facilis esse, incidunt culpa, omnis ipsum voluptatibus nostrum maiores suscipit veritatis, ipsam aliquam nesciunt voluptates molestias reiciendis illum nobis. Molestiae, odio voluptates.




```
<script>
    const variable = [];
    variable.push(12);
    console.log(variable);
</script>
```

En revanche, si “const” est un tableau, on peut modifier le contenu du tableau,
Car techniquement, on a pas modifier sa valeur qui est “tableau”….
La fonction .push() va ajouter la valeur entre parenthèse au tableau.

Une autre notion importante, est celle de la portée d'une variable…
La « portée » d'une variable désigne l'espace du script dans laquelle elle va être accessible. Toutes nos variables ne sont pas automatiquement disponibles à n'importe quel endroit dans un script et on ne va donc pas toujours pouvoir les utiliser. Cela dépend d'où la variable a été initialisée.

Algorithmie - Les variables

```
<script>
    //Variable globale car disponible dans tout le document
    let variable="hey !";

    //Bloc conditionnel
    if(1==1){
        //variable locale, disponible UNIQUEMENT dans le bloc conditionnel
        let variable2="oh ! ";

        console.log('Disponible dans le bloc conditionnel : '+variable);
        console.log('Disponible dans le bloc conditionnel : '+variable2);
    }

    console.log('Disponible dans la page : '+variable);
    console.log('Disponible dans la page : '+variable2);
</script>
```

La variable “variable” est disponible partout dans le script, car initialisée hors bloc (on verra plus en détail plus tard) et en haut de page. (Je ne peux pas utiliser cette variable au dessus de son initialisation). On dit qu’elle est **globale**.

La variable “variable2” n’est disponible uniquement dans le bloc conditionnel (entre les accolades), car elle a été initialisée dans ce bloc. Je ne peux pas l’afficher en dehors. On dit qu’elle est **locale**.

Algorithmie - Les variables

Lorem ipsum dolor sit amet consectetur adipisicing elit. Facilis esse, incidunt culpa, omnis ipsum voluptatibus nostrum maiores suscipit veritatis, ipsam aliquam nesciunt voluptates molestias reiciendis illum nobis. Molestiae, odio voluptates.

```
<script>
    //Variable globale car disponible dans tout le document
    let variable="hey !";

    //Bloc conditionnel
    if(1==1){
        //variable locale, disponible UNIQUEMENT dans le bloc conditionnel
        let variable2="oh ! ";

        console.log('Disponible dans le bloc conditionnel : '+variable);
        console.log('Disponible dans le bloc conditionnel : '+variable2);
    }

    console.log('Disponible dans la page : '+variable);
    console.log('Disponible dans la page : '+variable2);
</script>
```



Normalement, je devrais avoir 4 messages. 2 commençant par “Disponible dans le bloc conditionnel” et 2 commençant par “Disponible dans la page”.

Mais comme je ne peux pas utiliser “variable2” en dehors du bloc conditionnel (et que j’essaie de l’afficher quand même), le JS me génère une erreur (en me disant que “variable2” n’est pas définie).

Algorithmie - Les variables

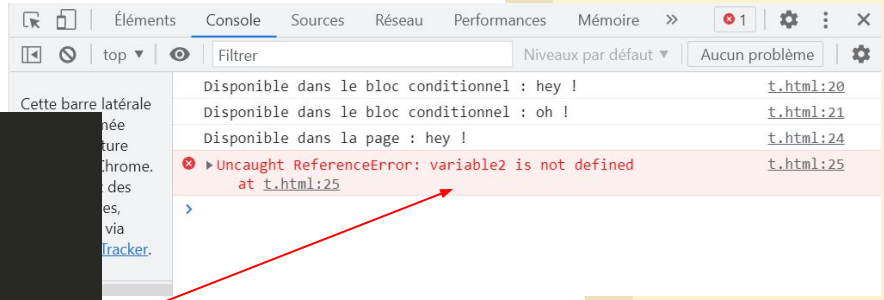
Lorem ipsum dolor sit amet consectetur adipisicing elit. Facilis esse, incidunt culpa, omnis ipsum voluptatibus nostrum maiores suscipit veritatis, ipsam aliquam nesciunt voluptates molestias reiciendis illum nobis. Molestiae, odio voluptates.

```
<script>
    //Variable globale car disponible dans tout le document
    const variable="hey !";

    //Bloc conditionnel
    if(1==1){
        //variable locale, disponible UNIQUEMENT dans le bloc conditionnel
        const variable2="oh ! ";

        console.log('Disponible dans le bloc conditionnel : '+variable);
        console.log('Disponible dans le bloc conditionnel : '+variable2);
    }

    console.log('Disponible dans la page : '+variable);
    console.log('Disponible dans la page : '+variable2);
</script>
```

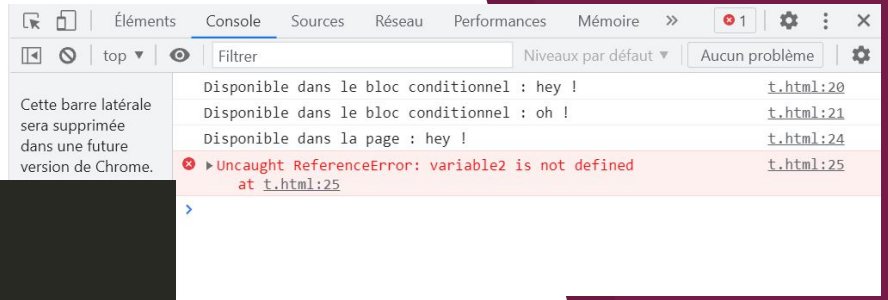


Ce qui est valable pour “let” l’est aussi pour “const”

Autre notion, la concaténation. Concaténer signifie littéralement « mettre bout à bout ». La concaténation est un mot généralement utilisé pour désigner le fait de rassembler deux chaînes de caractères pour en former une nouvelle. La concaténation va nous permettre de mettre bout-à-bout une chaîne de caractères et la valeur d'une variable par exemple.

Algorithmie - Les variables

Lorem ipsum dolor sit amet consectetur adipisicing elit. Facilis esse, incidunt culpa, omnis ipsum voluptatibus nostrum maiores suscipit veritatis, ipsam aliquam nesciunt voluptates molestias reiciendis illum nobis. Molestiae, odio voluptates.




```
<script>
    //Variable globale car disponible dans tout le document
    const variable="hey !";

    //Bloc conditionnel
    if(1==1){
        //variable locale, disponible UNIQUEMENT dans le bloc conditionnel
        const variable2="oh ! ";

        console.log('Disponible dans le bloc conditionnel : '+variable);
        console.log('Disponible dans le bloc conditionnel : '+variable2);
    }

    console.log('Disponible dans la page : '+variable);
    console.log('Disponible dans la page : '+variable2);
</script>
```

D'ailleurs, nous l'avons vu dans les exercices précédents…

En JavaScript, l'opérateur de concaténation est le signe +. Faites bien attention ici : lorsque le signe + est utilisé avec deux nombres, il sert à les additionner. Lorsqu'il est utilisé avec autre chose que deux nombres, il sert d'opérateur de concaténation.

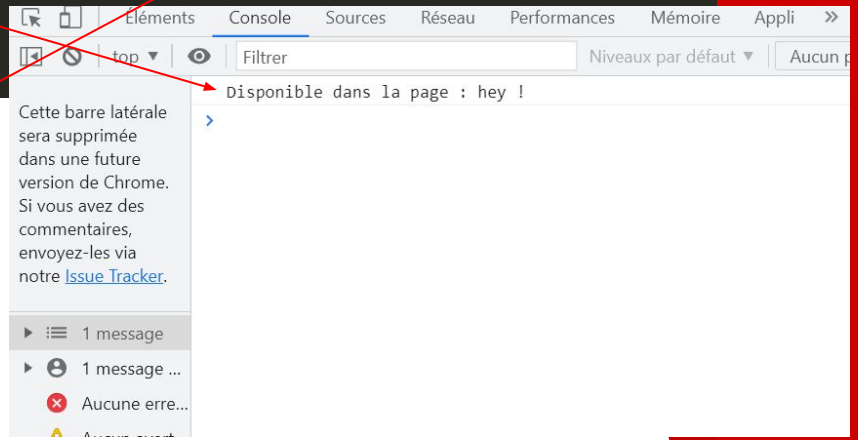
Algorithmie - Les variables

```
<script>

    const variable="hey !";

    console.log('Disponible dans la page : '+variable);

</script>
```



J'ai allégé un peu le code. On voit que la chaîne de caractère est entre les guillemets et que la variable est après le signe +

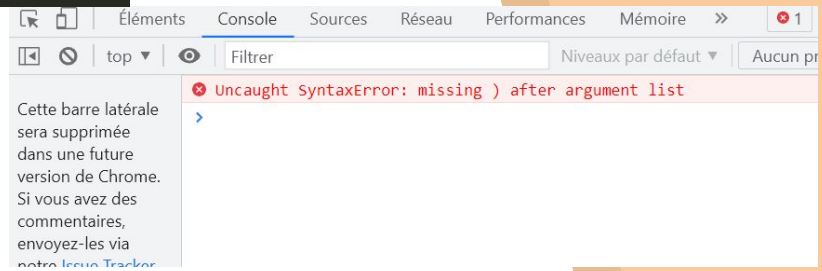
Algorithmie - Les variables

```
<script>

    const variable="hey !";

    console.log('Disponible dans la page : 'variable);

</script>
```



Si j'oublie le signe +, nous avons une erreur...

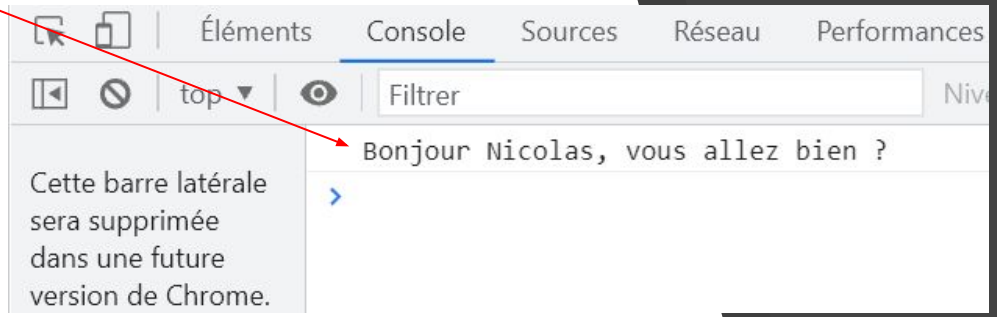
Algorithmie - Les variables

```
<script>

    const variable="Nicolas";

    console.log('Bonjour '+variable+', vous allez bien ?');

</script>
ody>
```



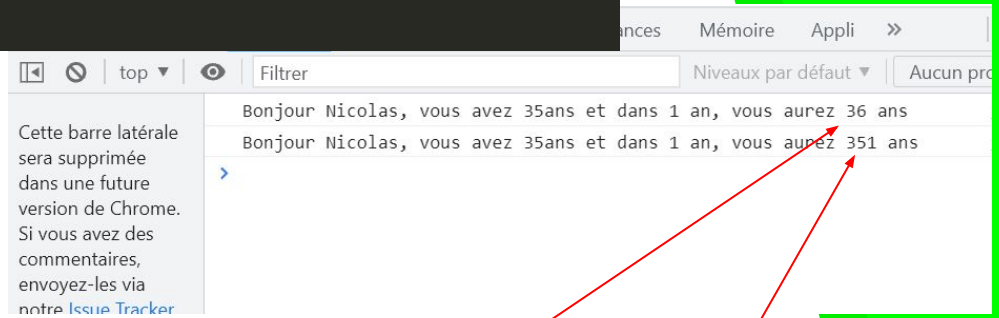
On peut également intercaler une variable entre 2 chaînes de caractère

Algorithmie - Les variables

```
const age = 35;
const nom ="Nicolas";
const text = 'Bonjour '+nom+', vous avez '+ age +'ans et dans 1 an, vous aurez '+ (age+1)+' ans';
const text2 = 'Bonjour '+nom+', vous avez '+ age +'ans et dans 1 an, vous aurez '+ age+1 +' ans';

console.log(text);
console.log(text2);

script>
```



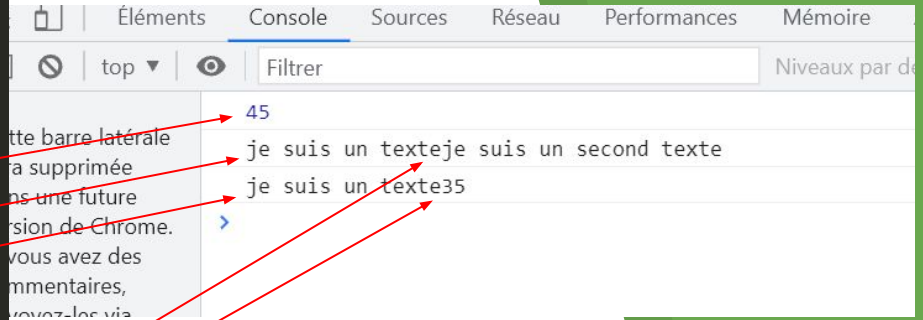
Dans l'exemple ci-dessus, nous voyons que JS peut soit concaténer (text2, il a collé 35 et 1) soit additionner (text). Le fait d'avoir écrit (age+1), JS comprend qu'il doit les additionner car "age" est un chiffre comme 1.

Dans "text2" le JS a "chaine de caractère (...vous aurez) + chiffre (35) + chiffre (1) + chaine de caractère (an)" . Comme il ne peut pas additionner une chaîne de caractère avec des nombre, il les concatènent

Algorithmie - Les variables

```
const var1 = 35;
const var2 = 10;
let var3 ="je suis un texte";
let var4 = "je suis un second texte";

console.log(var1 + var2);
console.log(var3 + var4);
console.log(var3 + var1);
/script>
```



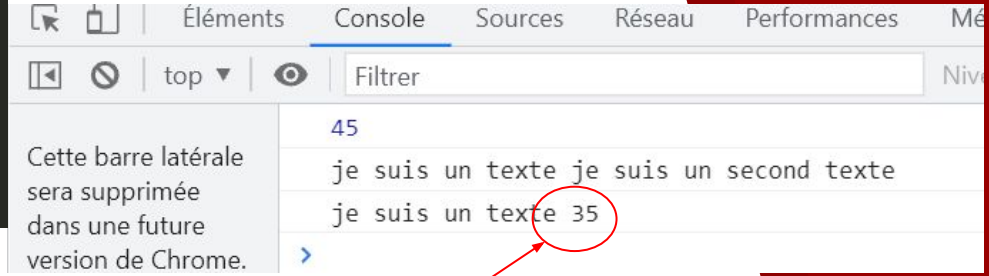
Dans le premier cas, JS additionne les 2 variables car ce sont 2 nombre
Dans le deuxième cas, JS les concatène car ce sont 2 chaînes de caractères
Dans le derniers cas, JS les concatène car on ne peut pas additionner un chiffre et chaîne de caractères.

Petite parenthèse, on voit qu'il n'y a pas d'espace entre les 2 variables concaténées.

Algorithmie - Les variables

```
  const var1 = 35;
  const var2 = 10;
  let var3 ="je suis un texte";
  let var4 = "je suis un second texte";

  console.log(var1 + var2);
  console.log(var3 +' '+ var4);
  console.log(var3 +' '+ var1);
script>
```



Pour mettre un espace entre 2 variables, il faut mettre une chaîne de caractère vide (un espace entre 2 guillemets)

Et maintenant voyons….les littéraux de gabarits !

Algorithmie - Les variables

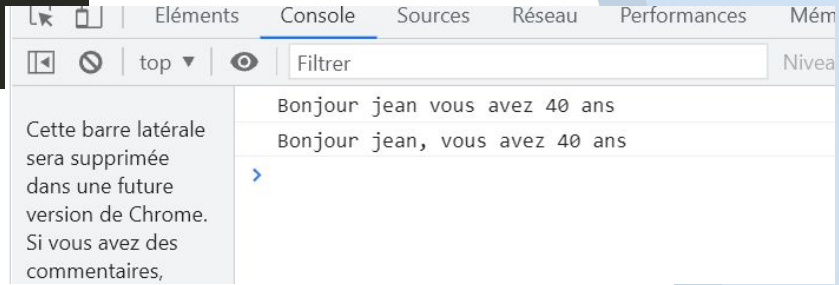
```
<script>

    const nom = 'jean';
    const age = 40;


    console.log('Bonjour '+nom+ ' vous avez '+age+ ' ans');

    console.log(`Bonjour ${nom}, vous avez ${age} ans`);

</script>
</body>
```



Il existe une méthode plus récente pour la mise en forme d'un texte et la concaténation.
Pour ce faire, on ne met pas le texte en guillemets (simple ou double), mais entre "accents graves". Et de faire passer les variable dans une syntaxe particulière ${ma_variable} mais plus besoin du signe +.La variable est directement intégrée dans la chaîne de caractère.

Algorithmie - Les variables

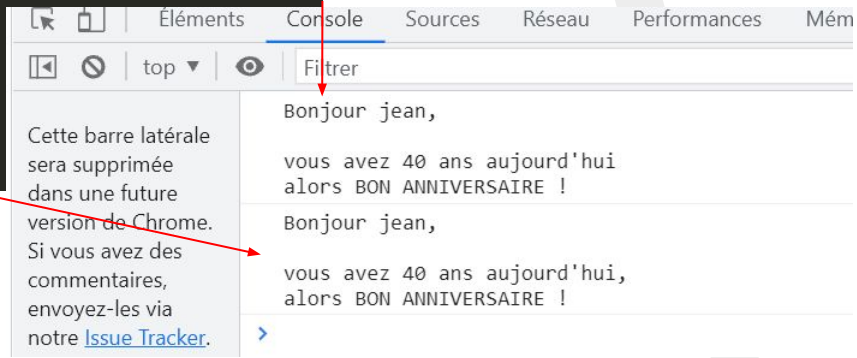
```
const nom = 'jean';
const age = 40;

const message = 'Bonjour '+nom+ ',\n\nvous avez '+age+ ' ans aujourd\'hui\nalors BON ANNIVERSAIRE !';

const message2 =
`Bonjour ${nom},

vous avez ${age} ans aujourd'hui,
alors BON ANNIVERSAIRE !`;

console.log(message);
console.log(message2);
```



De plus, les littéraux de gabarits gèrent les sauts de ligne, alors que sans cette technique, j'ai besoin d'insérer \n (qui signifie saut de ligne).

Du coup, dans mon code, la variable "message2" est plus facilement lisible que "message"...

Algorithmie - Les variables

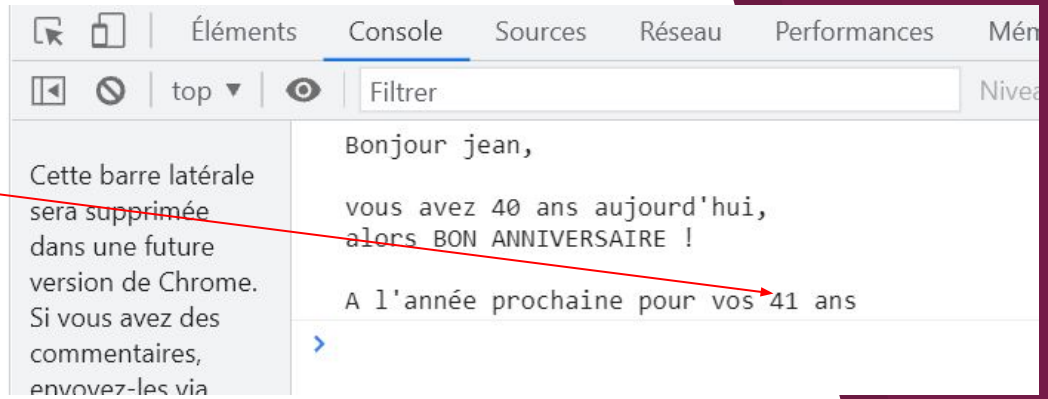
```
const nom = 'jean';
const age = 40;


const message2 =
`Bonjour ${nom},

vous avez ${age} ans aujourd'hui,
alors BON ANNIVERSAIRE !

A l'année prochaine pour vos ${age + 1} ans`;


console.log(message2);
```



Bien entendu, on peut également passer des calculs mathématique à l'intérieur des accolades….

Algorithmie,
Les Opérateurs

Algorithmie - Les Opérateurs

Un opérateur est un symbole qui va être utilisé pour effectuer certaines actions notamment sur les variables et leurs valeurs.

Il existe différents types d'opérateurs qui vont nous servir à réaliser des opérations de types différents. Les plus fréquemment utilisés sont :

- Les opérateurs arithmétiques ;
- Les opérateurs d'affectation / d'assignation ;
- Les opérateurs de comparaison ;
- Les opérateurs d'incrémentation et décrémentation ;
- Les opérateurs logiques ;
- L'opérateur de concaténation (vu plus haut) ;
- L'opérateur ternaire ;
- ...

Algorithmie - Les Opérateurs

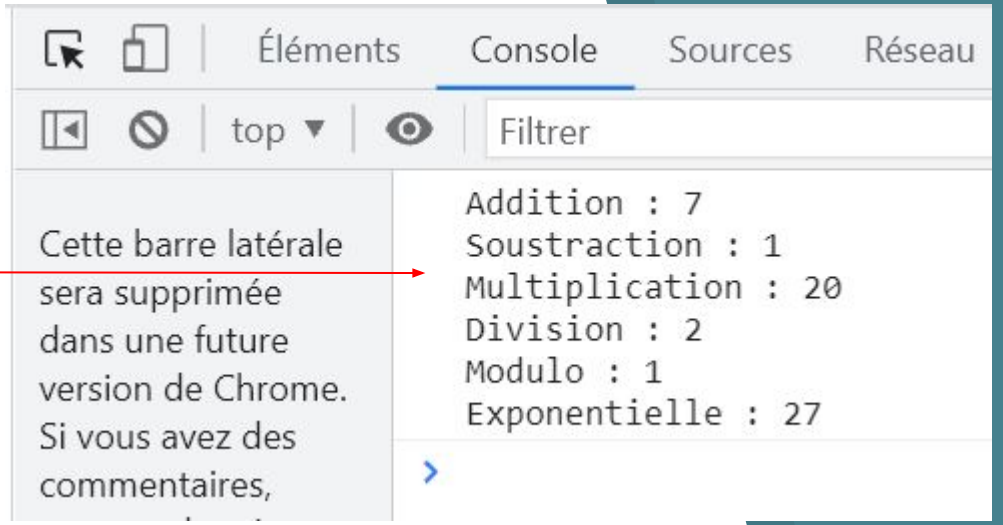
Opérateur	Nom de l'opération associée
+	Addition
–	Soustraction
*	Multiplication
/	Division
%	Modulo (reste d'une division euclidienne)
**	Exponentielle (élévation à la puissance d'un nombre par un autre)

Voici les opérateurs arithmétiques...

Algorithmie - Les Opérateurs

```
const num = 2;
const num2 = 3;
const num3 = 4;


const result =
`Addition : ${num3 + num2}
Soustraction : ${num2 - num}
Multiplication : ${num * 10}
Division : ${num3 / num}
Modulo : ${num3 % num2}
Exponentielle : ${num2**3}
`;
```

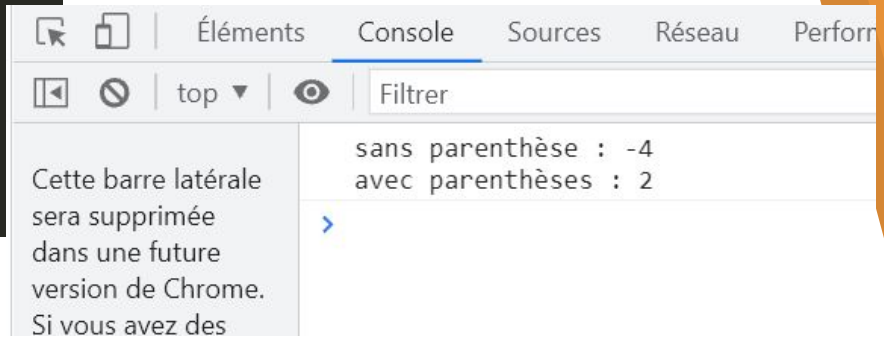


En action… Pour rappel, le modulo c'est le reste d'une division.

Algorithmie - Les Opérateurs

```
const result =
`sans parenthèse : ${1 - 2 - 3}
avec parenthèses : ${1 -(2 - 3)}
`;


console.log(result);
```



Il est possible de faire passer des priorité de calcul via des parenthèses.
Sans parenthèse, JS exécute le calcul de gauche à droite (1- 2 = -1 puis -1 - 3 = -4)
Avec parenthèse, JS exécute le calcule entre les parenthèses d'abord (2-3 = -1 puis 1- -1 = 2)

Algorithmie - Les Opérateurs

Opérateur	Définition
+=	Additionne puis affecte le résultat
-=	Soustrait puis affecte le résultat
*=	Multiplie puis affecte le résultat
/=	Divise puis affecte le résultat
%=	Calcule le modulo puis affecte le résultat

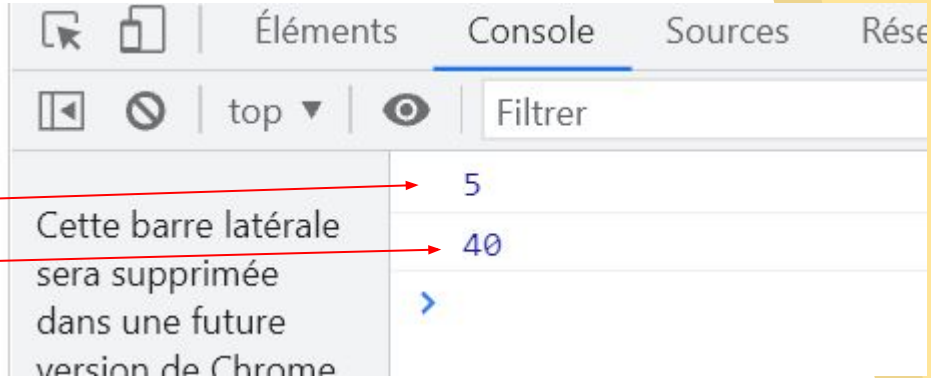
Voici les opérateurs d’affectation / d’assignation

Algorithmie - Les Opérateurs

```
let num = 2;
let num2 = 3;
let num3 = 4;


num += num2;
num3 *= 10;


console.log(num);
console.log(num3);
```



Il faut les voir comme un raccourci :
“num” est égal à 2, puis on additionne “num” (donc 2) à “num2” (soit 3) et on stock le résultat dans “num”.
“num += num2” c’est comme “num = num + num2”
“num3 *= 10” c’est comme “num3 = num3 * 10”
D’ailleurs le signe = est également un opérateur d’assignation.

Algorithmie - Les Opérateurs

Opérateur	Définition
==	Permet de tester l'égalité sur les valeurs
===	Permet de tester l'égalité en termes de valeurs et de types
!=	Permet de tester la différence en valeurs
<>	Permet également de tester la différence en valeurs
!==	Permet de tester la différence en valeurs ou en types
<	Permet de tester si une valeur est strictement inférieure à une autre
>	Permet de tester si une valeur est strictement supérieure à une autre
<=	Permet de tester si une valeur est inférieure ou égale à une autre
>=	Permet de tester si une valeur est supérieure ou égale à une autre

Voici les opérateurs de comparaison

Algorithmie - Les Opérateurs

Opérateur	Définition
==	Permet de tester l'égalité sur les valeurs
===	Permet de tester l'égalité en termes de valeurs et de types
!=	Permet de tester la différence en valeurs
<>	Permet également de tester la différence en valeurs
!==	Permet de tester la différence en valeurs ou en types
<	Permet de tester si une valeur est strictement inférieure à une autre
>	Permet de tester si une valeur est strictement supérieure à une autre
<=	Permet de tester si une valeur est inférieure ou égale à une autre
>=	Permet de tester si une valeur est supérieure ou égale à une autre

Certains opérateurs vous sont déjà familier, on ne va pas revenir dessus…
Nous en reparlerons au moment des conditions.

Algorithmie - Les Opérateurs

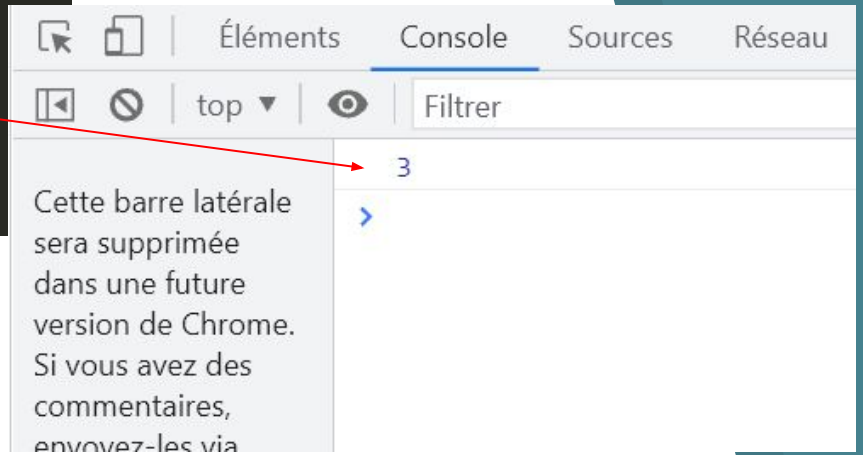
Exemple (opérateur + variable)	Résultat
`++x`	Pré-incrémentation : incrémente la valeur contenue dans la variable x, puis retourne la valeur incrémentée
`x++`	Post-incrémentation : retourne la valeur contenue dans x avant incrémentation, puis incrémente la valeur de $x
`--x`	Pré-décrémentation : décrémente la valeur contenue dans la variable x, puis retourne la valeur décrémentée
`x--`	Post-décrémentation : retourne la valeur contenue dans x avant décrémentation, puis décrémente la valeur de $x

Les opérateurs d’incrémentation et décrémentation

Algorithmie - Les Opérateurs

```
let num = 2;
let num2 = 3;
let num3 = 4;


console.log(++num); // num = num + 1;
```

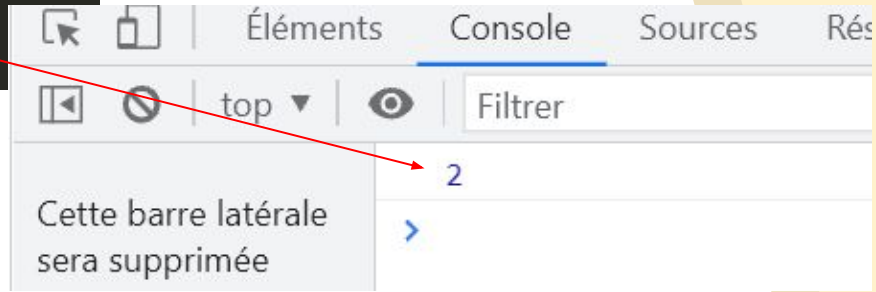


Le fait de mettre ++ avant le chiffre on dit a JS “d’abord tu incrémente de 1 et ensuite tu m’affiches le résultat).

Algorithmie - Les Opérateurs

```
let num = 2;
let num2 = 3;
let num3 = 4;


console.log(num++); // num = num + 1;
```

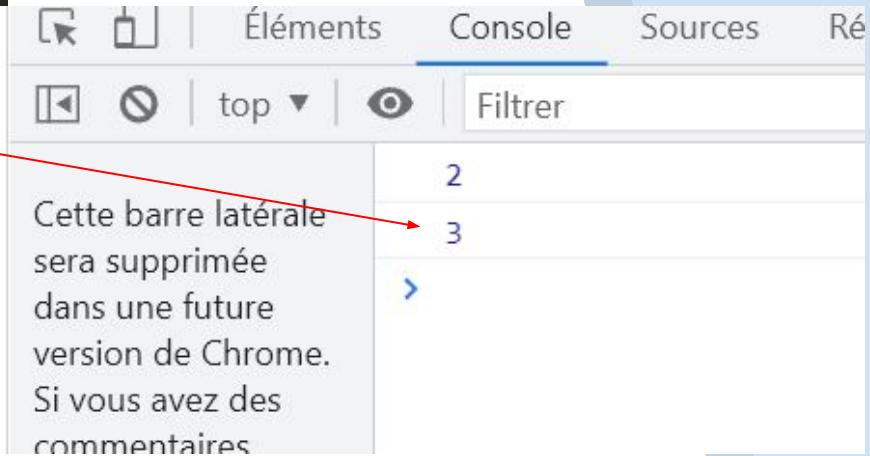


Le fait de mettre ++ après le chiffre on dit a JS “d’abord tu m’affiches le résultat PUIS tu incrémentes de 1).

Algorithmie - Les Opérateurs

```
let num = 2;
let num2 = 3;
let num3 = 4;


console.log(num++); // num = num + 1;
console.log(num);
```



D'ailleurs, si je décide de ré-afficher la variable "num" après l'incrémentation, on voit qu'elle a bien augmenté de 1. Pas de slide pour le -- (décrémentation) ou -1 mais c'est le même combat que ++.

Algorithmie - Les Opérateurs

Opérateur (nom)	Opérateur (symbole)	Description
AND (ET)	&&	Lorsqu'il est utilisé avec des valeurs booléennes, renvoie true si toutes les comparaisons sont évaluées à true ou false sinon
OR (OU)	||	Lorsqu'il est utilisé avec des valeurs booléennes, renvoie true si au moins l'une des comparaisons est évaluée à true ou false sinon
NO (NON)	!	Renvoie false si une comparaison est évaluée à true ou renvoie true dans le cas contraire

Les opérateurs de logique (que nous verrons plus en détails dans la partie des conditions) comme les opérateurs ternaires…(var ? 'un truc' : 'un autre truc')

Algorithmie, Les Conditions

Algorithmie - Les Conditions

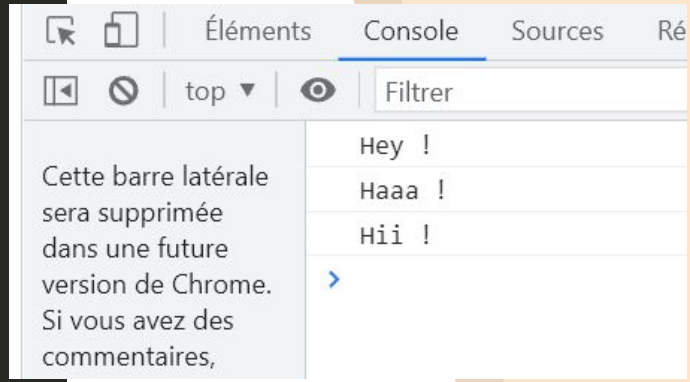
```
if(ma_condition_est vraie){
    console.log(alors_jaffiche_ce_message);
}
```

La condition if est l'une des conditions les plus utilisées et est également la plus simple à appréhender puisqu'elle va juste nous permettre d'exécuter un bloc de code si et seulement si le résultat d'un test vaut true.

Algorithmie - Les Conditions

```
let num = 2;
let num2 = 3;
let num3 = 4;


//Si num est égal à 2
//VRAI
if(num == 2){
    console.log('Hey !');
}
//Si num est égal à num2 (si 2 est égal à 3)
//FAUX
if(num == num2){
    console.log('Hooo !');
}
//Si num est plus petit que num2 (si 2 est plus petit que 3)
//VRAI
if(num < num2){
    console.log('Haaa !');
}
//Si num est plus grand OU égal à num2 (si 2 est plus grand OU égal à 3)
//VRAI
if(num >= 2){
    console.log('Hii !');
}
```



Voici des exemples simples de conditions. A chaque fois que ce qu'il y a entre parenthèses est vrai, ce qu'il y a entre les accolades sera exécuté ! Dans l'exemple ci-dessus la seule condition fausse est la deuxième, "Hooo !" n'est pas affiché.

Algorithmie - Les Conditions

```
let num = 2;
let num2 = 3;
let num3 = 4;


//Si num est égal à 2
//VRAI
if(num == 2){
    console.log('Hey !');
}
//Si num est égal à num2 (si 2 est égal à 3)
//FAUX
if(num == num2){
    console.log('Hooo !');
}
//Si num est plus petit que num2 (si 2 est plus petit que 3)
//VRAI
if(num < num2){
    console.log('Haaa !');
}
//Si num est plus grand OU égal à num2 (si 2 est plus grand OU égal à 3)
//VRAI
if(num >= 2){
    console.log('Hii !');
}
```

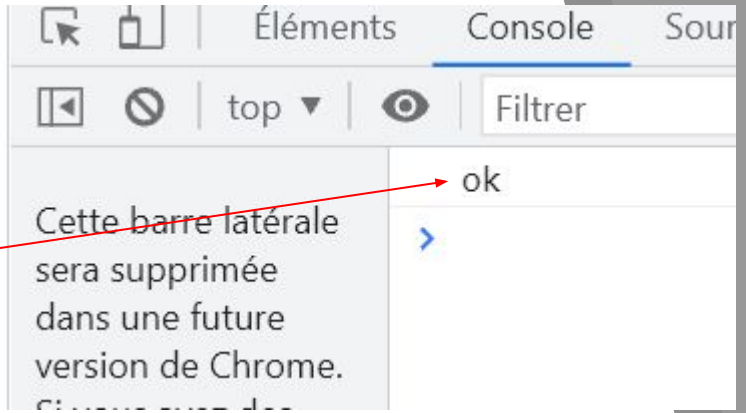
Notons que pour comparer, on utilise == et non =. Car = est un opérateur d'assignation et non de comparaison (ex : let variable = 12; On assigne la valeur de 12 à la variable "variable").

Algorithmie - Les Conditions

```
let num = 2;
let num2 = 3;
let num3 = 4;
let var1 ="";

if(num){
    console.log('ok');
}

if(var1){
    console.log('coucou');
}
```

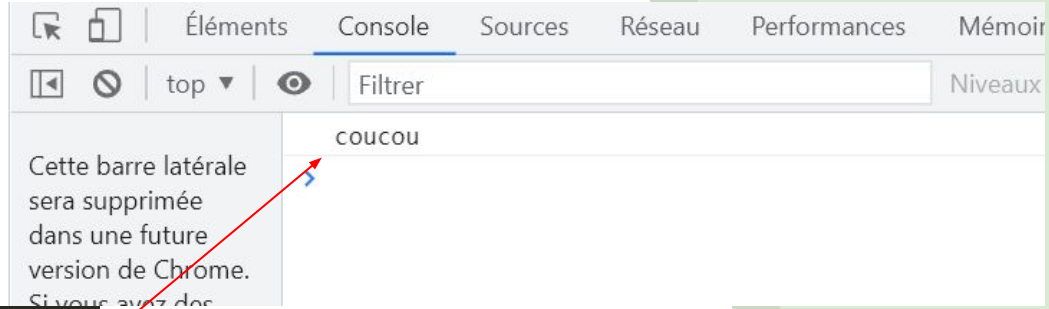


Si aucun opérateur (==,>,>+,...) n'est présent dans la condition "if", JS va traduire ça par "est-ce que "num" existe…. Comme "num" est égal à 2, elle existe.
En revanche la variable "var1" est égal à "" (soit rien), JS considère qu'elle n'existe pas (ce qui est logique car par définition "rien" n'existe pas).

Algorithmie - Les Conditions

```
let var1 =" ";


if(var1){
    console.log('coucou');
}
```

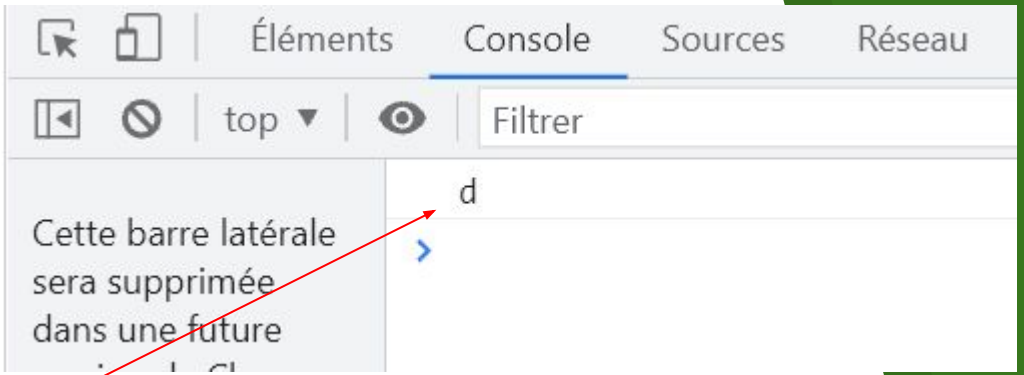


Par contre, si “var1” est égal à un espace, JS considère que ce n’est pas “rien”, donc le code s’exécute

Algorithmie - Les Conditions

```
let a = false;
let b = 0;
let c = '';
let d = '.'

if(a){
    console.log('a');
}
if(b){
    console.log('b');
}
if(c){
    console.log('c');
}
if(d){
    console.log('d');
}
```

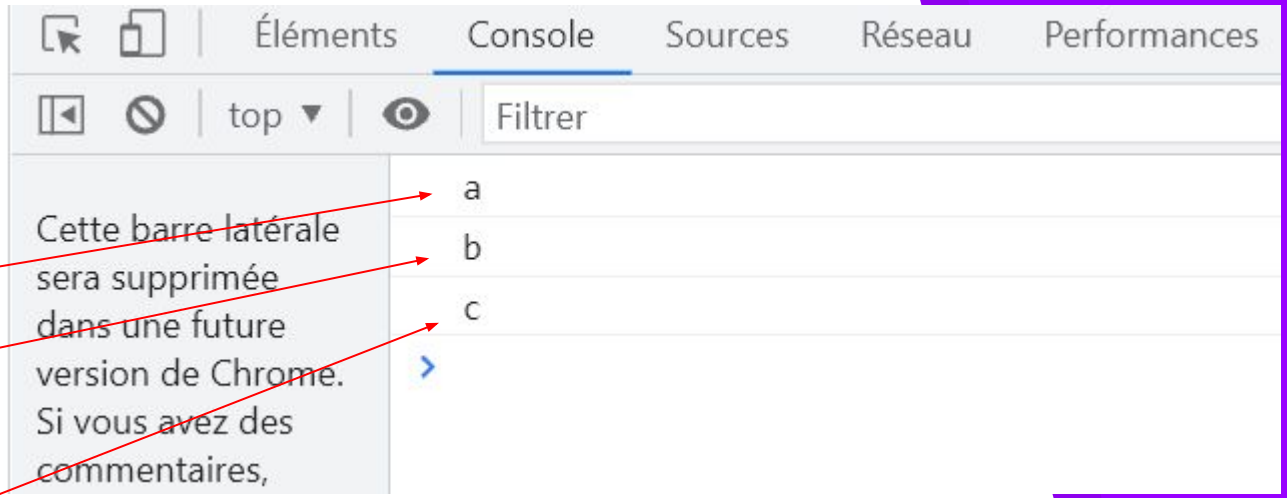


Considérer comme vide / nul : False; 0; null; undefined; NaN (« Not a Number » = « n'est pas un nombre »); '' (chaine de caractère vide)

Algorithmie - Les Conditions

```
let a = false;
let b = 0;
let c = '';
let d = '.'

if(!a){
    console.log('a');
}
if(!b){
    console.log('b');
}
if(!c){
    console.log('c');
}
if(!d){
    console.log('d');
}
```

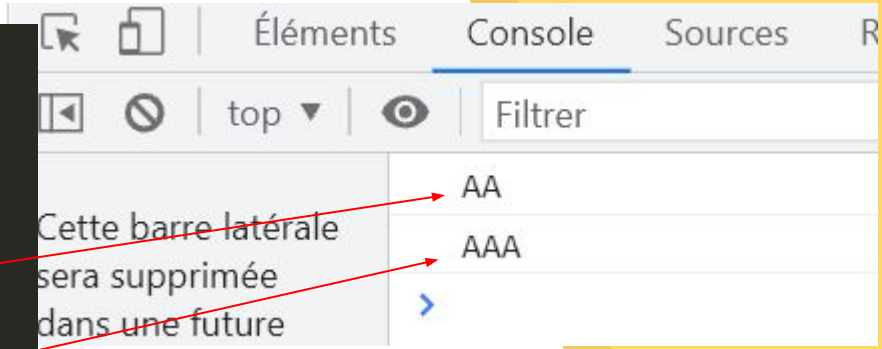


L'opérateur "!" signifie "pas". JS traduit ça par "si la variable n'existe pas, alors j'exécute le code. C'est le comportement inverse.

Algorithmie - Les Conditions

```
let a = '';

//3 n'est pas égal à 4 donc VRAI puis vrai est égal à faux donc FAUX
//FAUX
if((3 != 4) == false){
 console.log('A');
}
//2 n'est pas égal à 2 c'est FAUX puis faux est égal à faux c'est VRAI
//VRAI
if((2!=2) == false){
    console.log('AA');
}
//a est égal à rien (donc faux) donc faux est égal à faux, c'est VRAI
//VRAI
if(a==false){
    console.log('AAA');
}
```

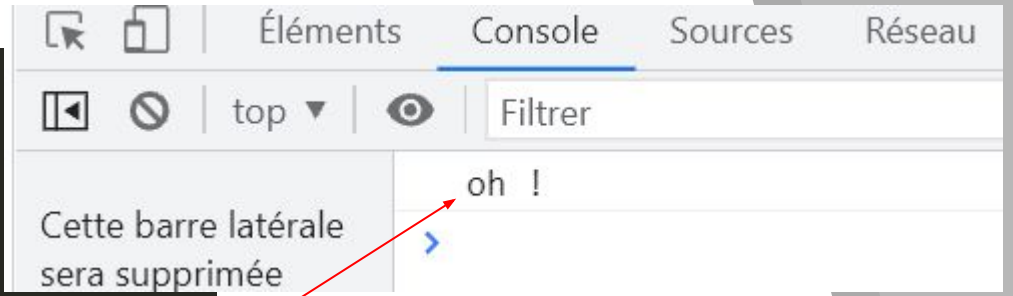


Comme pour l'arithmétique, on peut utiliser les parenthèses pour renverser la valeur logique (! fait très bien l'affaire mais c'est pour exemple)

Algorithmie - Les Conditions

```
let num = 2;
let num2 = 3;
let num3 = 4;

if(num != 3){
    console.log('oh !');
}
```



Dans cet exemple, on dit au JS, “si la variable ‘num’ n’est pas égal à 3, alors exécute le code”. (donc si 2 n’est pas égal à 3). Attention, cela ne fonctionne pas avec un autre opérateur que = (ex : ~~!>~~)

Algorithmie - Les Conditions

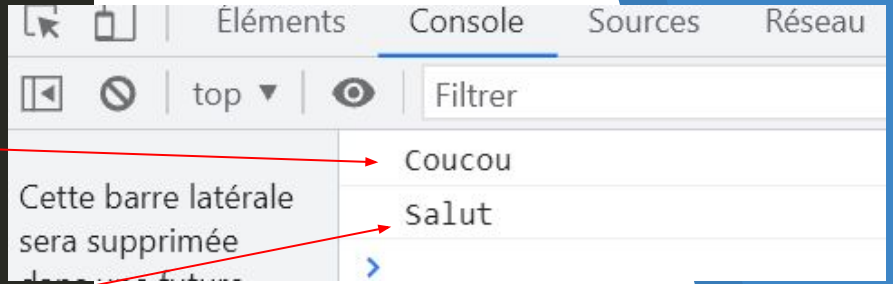
Opérateur	Définition
==	Permet de tester l'égalité sur les valeurs
===	Permet de tester l'égalité en termes de valeurs et de types
!=	Permet de tester la différence en valeurs
<>	Permet également de tester la différence en valeurs
!==	Permet de tester la différence en valeurs ou en types
<	Permet de tester si une valeur est strictement inférieure à une autre
>	Permet de tester si une valeur est strictement supérieure à une autre
<=	Permet de tester si une valeur est inférieure ou égale à une autre
>=	Permet de tester si une valeur est supérieure ou égale à une autre

Il nous reste 3 opérateurs de comparaison à voir

Algorithmie - Les Conditions

```
//La variable 'a' est une chaine de caractère car entre guillemets
//Ce n'est pas un nombre !
let a = '12';

//a est-il égal à 12
//VRAI
if(a == 12){
    console.log('Coucou');
}
//a 12 est-il égal à 12 ET de type nombre ??
//FAUX
if(a === 12){
    console.log('Bonsoir');
}
//a 12 est-il égal à 12 ET de est-il une chaine de caractère ??
//VRAI
if(a === '12'){
    console.log('Salut');
}
```



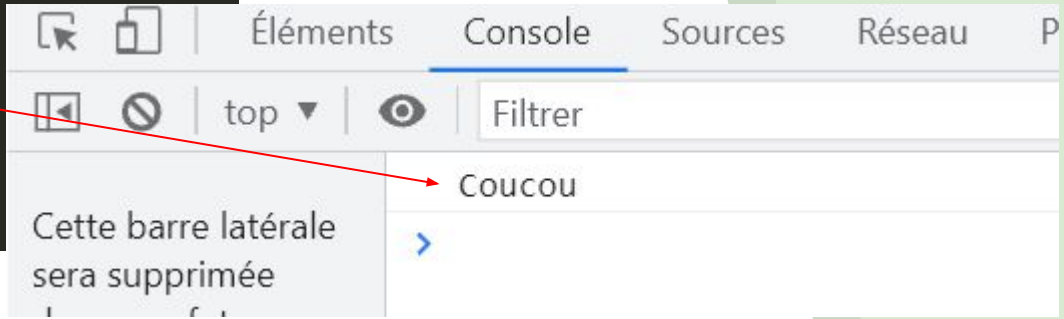
Le triple égal (===) vérifie la valeur ET le type de variable.
Il est plus strict que == qui ne vérifie que la valeur mais peut éviter certaines erreurs. Pour JS 12 et ‘12’ ont la même valeur, cependant pas le même type.

Algorithmie - Les Conditions

```
//La variable 'a' est une chaine de caractère car entre guillemets
//Ce n'est pas un nombre !
let a = '12';


if(a !== 12){
    console.log('Coucou');
}

if(a !== '12'){
    console.log('Salut');
}
```



Du coup le !== fait l'opposé de l'opérateur ===, il vérifie que la valeur ET le type sont bien différents.

Algorithmie - Les Conditions

Opérateur	Définition
==	Permet de tester l'égalité sur les valeurs
===	Permet de tester l'égalité en termes de valeurs et de types
!=	Permet de tester la différence en valeurs
<>	Permet également de tester la différence en valeurs
!==	Permet de tester la différence en valeurs ou en types
<	Permet de tester si une valeur est strictement inférieure à une autre
>	Permet de tester si une valeur est strictement supérieure à une autre
<=	Permet de tester si une valeur est inférieure ou égale à une autre
>=	Permet de tester si une valeur est supérieure ou égale à une autre

Cet opérateur ne fonctionne plus et je n'ai pas trouver de documentation le concernant. Lorsqu'il est utilisé, il provoque une erreur. C'est l'équivalent de l'opérateur !=

La condition if est une structure conditionnelle limitée par définition puisqu'elle ne nous permet d'exécuter un bloc de code que dans le cas où le résultat d'un test est évalué à true mais elle ne nous offre aucun support dans le cas contraire.

La structure conditionnelle if…else (« si… sinon » en français) va être plus complète que la condition if puisqu'elle va nous permettre d'exécuter un premier bloc de code si un test renvoie true ou un autre bloc de code dans le cas contraire.

Algorithmie - Les Conditions

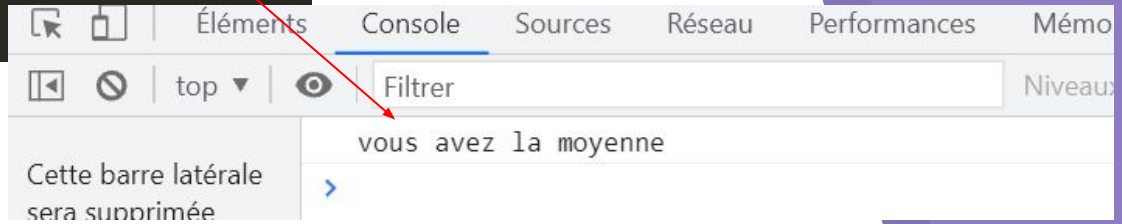
```
if(condition_est_vraie){
    console.log('ce code est éxécuté');
}else{
    console.log('sinon, ce code est éxécuté');
}
```

Si la condition entre les parenthèse est vraie, le premier code sera exécuté, sinon, ce sera le deuxième.

Algorithmie - Les Conditions

```
let note = 12;


if(note >= 10){
    console.log('vous avez la moyenne');
}else{
    console.log('vous n\'avez pas la moyenne');
}
```



Si la variable “note” est supérieur ou égal à 10, le code affichera ‘vous avez la moyenne’ sinon le code ‘affichera ‘vous n’avez pas la moyenne’ . Le “else” est un filet de sécurité qui sera forcément exécuter si la condition est fausse.

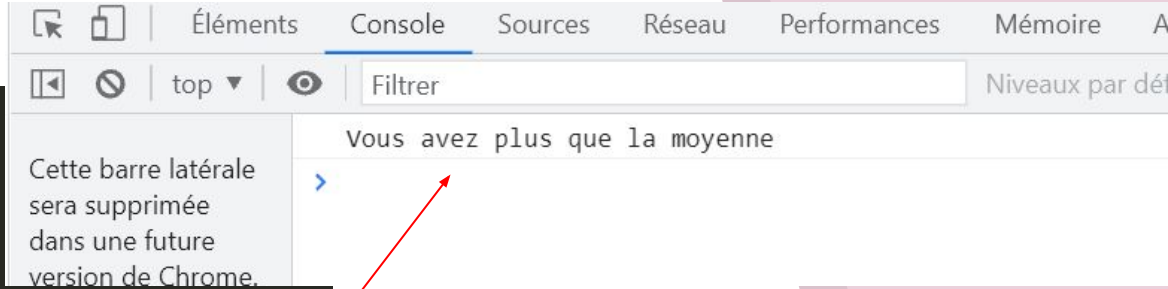
Algorithmie - Les Conditions

```let note = 12;


if(note > 10){

    if(note === 10){
        console.log('Vous avez tout juste la moyenne');
    }else{
        console.log('Vous avez plus que la moyenne');
    }

}else{
    console.log('vous n\'avez pas la moyenne');
}
```



Il est possible d'imbriquer des conditions dans des conditions mais cette syntaxe n'est pas vraiment facile à lire….

Algorithmie - Les Conditions

```
let note = 12;


if(note > 10){

    if(note === 10){
        console.log('Vous avez tout juste la moyenne');
    }else{
        console.log('Vous avez plus que la moyenne');
    }

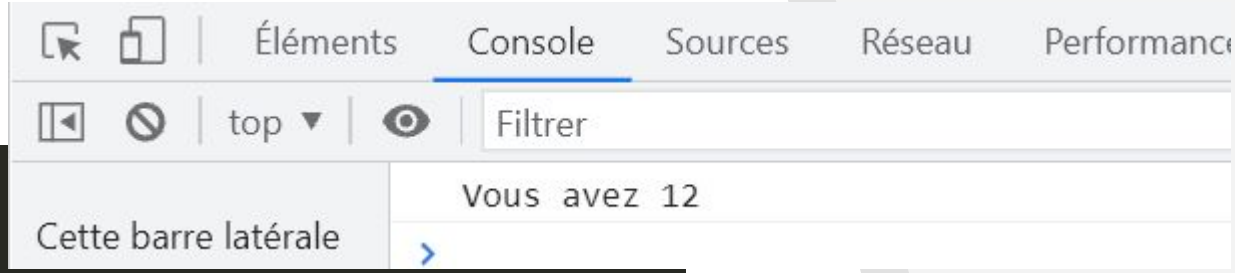
}else{
    console.log('vous n\'avez pas la moyenne');
}
```

```
let note = 12;


if(note == 10){
    console.log('vous avez tout juste la moyenne');
}else if(note > 10){
    console.log('vous avez plus que la moyenne');
}else{
 console.log('vous n\'avez pas la moyenne');
}
```

Pour faciliter la lecture du code, il existe le “if(condition 1){....}else if(condition 2){...}else{...}”

Algorithmie - Les Conditions

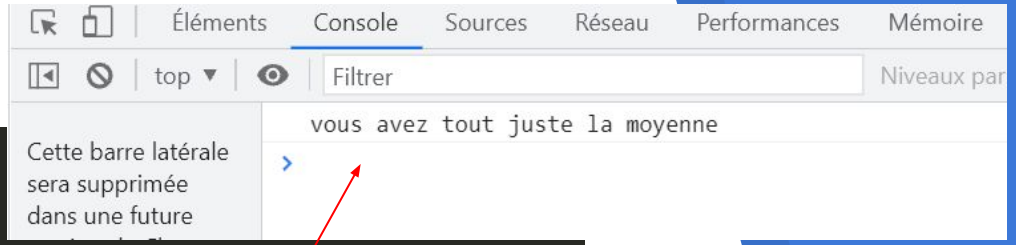



```
let note = 12;


if(note === 10){
    console.log('vous avez tout juste la moyenne');
}else if(note === 11){
    console.log(`Vous avez ${note}`);
}else if(note === 12){
    console.log(`Vous avez ${note}`);
}else{
    console.log('Vous avez soit moins de 10, soit plus de 12');
}
```

Vous pouvez chaîner autant de conditions que vous le souhaitez.

Algorithmie - Les Conditions




```
let note = 12;


if(note > 10){
    console.log('vous avez tout juste la moyenne');
}else if(note === 11){
    console.log(`Vous avez ${note}`);
}else if(note === 12){
    console.log(`Vous avez ${note}`);
}else{
    console.log('Vous avez soit moins de 10, soit plus de 12');
}
```

Attention : Si vous avez plusieurs conditions vraies, JS s'arrêtera à la première ! Dans l'exemple ci-dessus, la première et la troisième sont vraies. JS va quand même s'arrêter à la première.

Algorithmie - Les Conditions

Notion du SWITCH :

Le switch va nous permettre d'exécuter un code en fonction de la valeur d'une variable. On va pouvoir gérer autant de situations ou de « cas » que l'on souhaite.
Le switch représente une alternative à l'utilisation d'un if…else if…else.

Cependant, ces deux types d'instructions ne sont pas strictement équivalentes puisque dans un switch chaque cas va être lié à une valeur précise. En effet, l'instruction switch ne supporte pas l'utilisation des opérateurs de supériorité ou d'infériorité.

Dans certaines (rares) situations, il va pouvoir être intéressant d'utiliser un switch plutôt qu'un if…else if…else car cette instruction peut rendre le code plus clair et légèrement plus rapide dans son exécution.

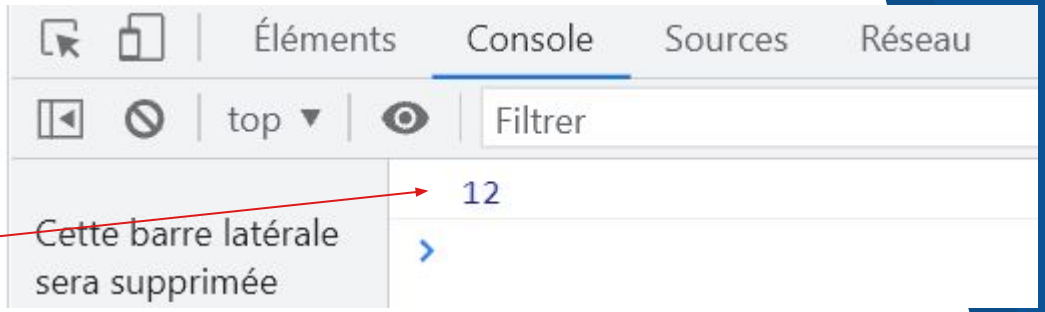
```
let note = 12;


switch (note) {
    case 10:
        console.log(10);
        break;
    case 11:
        console.log(11);
        break;
    case 12:
        console.log(12);
        break;
    case 13:
        console.log(13);
        break;
    default:
        console.log('nope');
        break;
}
```

Algorithmie - Les Conditions

```
let note = 12;

switch (note) {
    case 10:
        console.log(10);
        break;
    case 11:
        console.log(11);
        break;
    case 12:
        console.log(12);
        break;
    case 13:
        console.log(13);
        break;
    default:
        console.log('nope');
        break;
}
```



On vérifie la valeur entre les parenthèses du “switch”, puis, en fonction des cas (case), on exécute le code correspondant. A la fin de chaque cas (case), il y a un “break” pour stopper l’exécution du “switch”. A la fin de chaque “switch”, il y a un “default” (qui équivaut au “else”) pour afficher quelque chose au cas où aucun cas ne correspondent à la valeur du “switch”.

Les opérateurs logiques :

Les opérateurs logiques sont des opérateurs qui vont principalement être utilisés avec des valeurs booléennes et au sein de conditions.

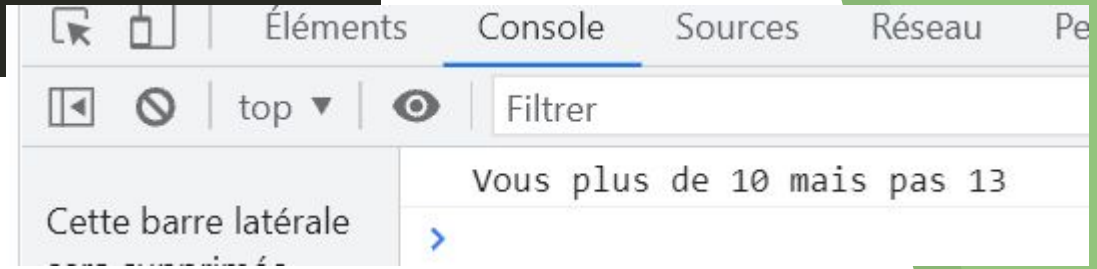
Le JavaScript supporte trois opérateurs logiques : l'opérateur logique « ET », l'opérateur logique « OU » et l'opérateur logique « NON ».

Les opérateurs logiques « ET » et « OU » vont nous permettre d'effectuer plusieurs comparaisons dans une condition. Si on utilise l'opérateur « ET », toutes les comparaisons devront être évaluées à true pour que le test global de la condition retourne true. Dans le cas où n'utilise l'opérateur logique « OU », il suffira qu'une seule des comparaisons soit évaluée à true pour exécuter le code dans la condition.

Algorithmie - Les Conditions

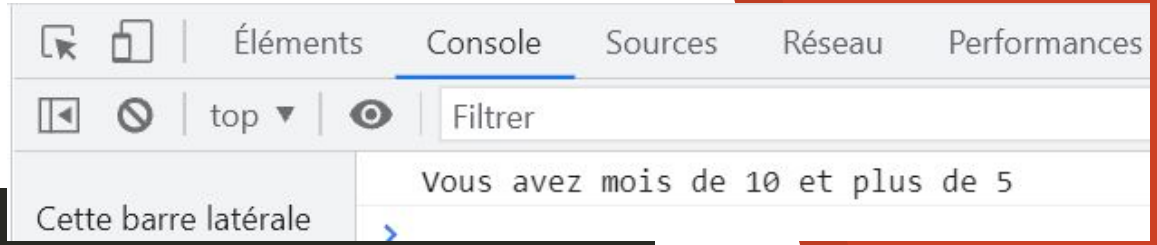
```
let note = 12;


if( note > 10 && note != 13){
    console.log('Vous avez plus de 10 mais pas 13');
}
```



La condition fonctionne uniquement si la note est strictement plus grande que 10 et n'est pas égale à 13...

Algorithmie - Les Conditions




```
let note = 7;


if( !(note > 10) && note > 5){
    console.log('Vous avez mois de 10 et plus de 5');
}
```

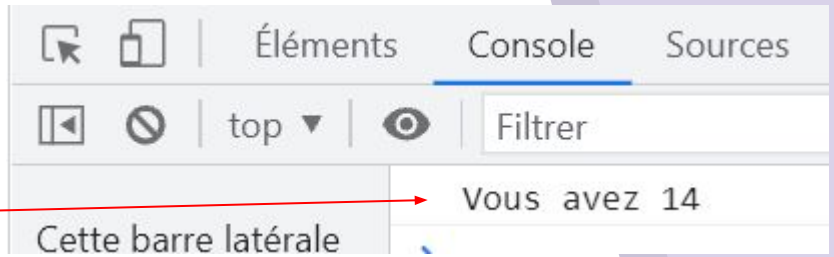
Si nous mettons des parenthèses autour d'une logique et un ! avant "!(logique)", on inverse cette logique. En clair, on demande au JS "si la note est plus petite que 10…"

Algorithmie - Les Conditions

```
let note = 7;


if( !(note > 10) || note == 14){
    console.log('Vous avez 14');
}

if( !(note > 10) && note == 14){
    console.log('Ooops !');
}
```



Dans la première condition, je demande au JS si ma note est plus petite que 10 OU si elle est égale à 14. Comme elle est inférieure à 10, le code s'exécute. Dans la deuxième, je lui dit "si la note est plus petite que 10 ET égale à 14…" ce qui est faux donc rien ne s'exécute...

Table de vérité

Vrai && Vrai = Vrai

Vrai && Faux = Faux

Faux && Faux = Faux

Vrai || Vrai = Vrai

Vrai || Faux = Vrai

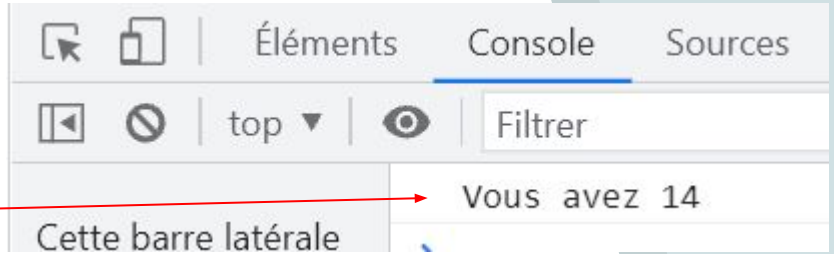
Faux || Faux = Faux

Algorithmie - Les Conditions

```
let note = 7;


if( !(note > 10) || note == 14){
    console.log('Vous avez 14');
}

if( !(note > 10) && note == 14){
    console.log('Ooops !');
}
```

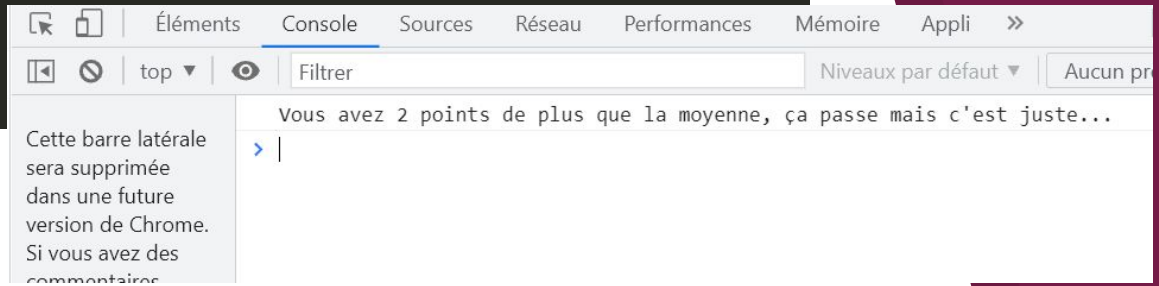


Première condition Vrai || Faux = Vrai
Deuxième condition Vrai && Faux = Faux

Algorithmie - Les Conditions

```
let note = 12;


//Si la note est plus petite que 10 ET plus grande que 5
if(note < 10 && note > 7){
    console.log(`Vous avez ${note}, vous êtes bon pour les rattrapages`);
//Si la note est strictement égal à 10
}else if(note===10){
    console.log(`Vous avez tout juste 10, ne vous ratez au prochain test !`);
//Si la note est plus grande que 10 ET plus petite que 13 (il a 11 ou 12 en fait...)
}else if(note > 10 && note < 13){
    console.log(`Vous avez ${note - 10} points de plus que la moyenne, ça passe mais c'est juste...`);
//Si la note est de 13 ou plus...
}else if(note > 12){
    console.log(`Vous avez géré ;)`);
}else{
    console.log('Vous allez redoubler :(');
}
```



On peut cumuler “else if” et opérateurs logiques...

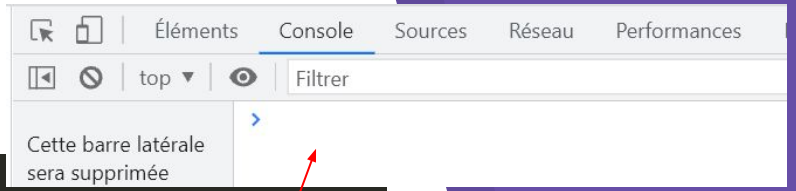
Algorithmie - Les Conditions

```
let note = 11;


if( !(note > 10) && ( note % 2 == 0  )  ){
    console.log('Serais-je exécuté ?');
}
```

D’après vous ??

Algorithmie - Les Conditions




```
let note = 11;

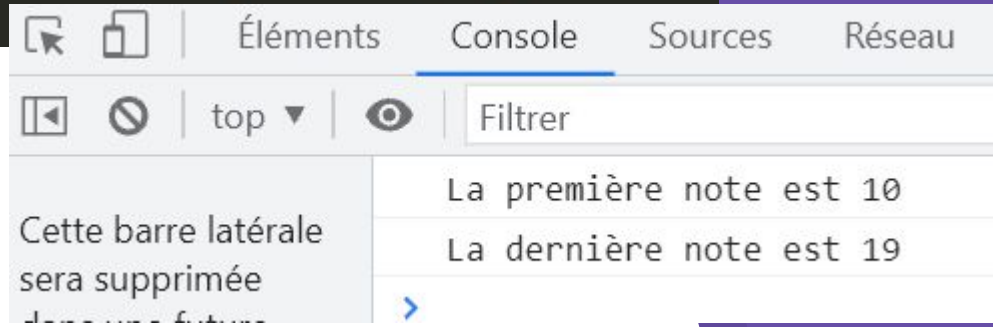

if( !(note > 10) && ( note % 2 == 0  )  ){
    console.log('Serais-je exécuté ?');
}
```

Et bien non…
Est-ce que note est plus petit que 10... non (Faux)
Est-ce que je peux diviser note par 2 et qu'il reste 0 (en gros, note est-il un nombre paire)...non (Faux)
Donc Faux && Faux = Faux

Algorithmie, Les Tableaux

Algorithmie - Les Tableaux

```
const notes = [10, 5, 14, 19];
console.log(`La première note est ${notes[0]}`);
console.log(`La dernière note est ${notes[3]}`);
```



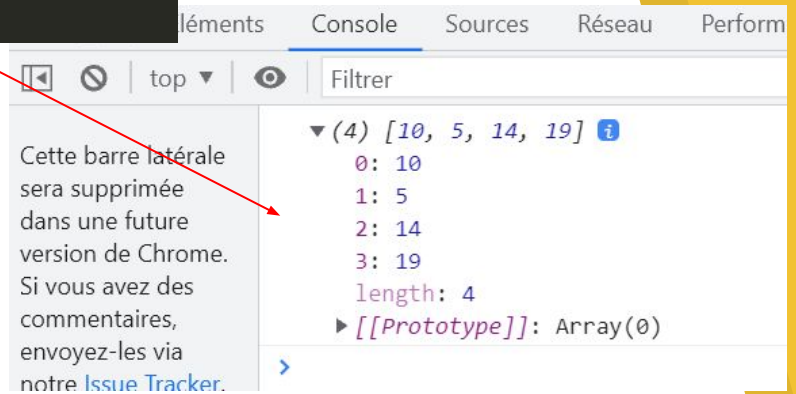
La variable “notes” est un tableau indexé.
“10” est en première position, ‘5’ en deuxième, etc…

Détail important, JS commence à compter à partir de 0. Donc le premier élément à l’index 0 et pas 1.

notes[0] // On demande la valeur 1 du tableau “notes” -> 10
notes[3] // On demande la valeur 4 du tableau “notes” -> 19

Algorithmie - Les Tableaux

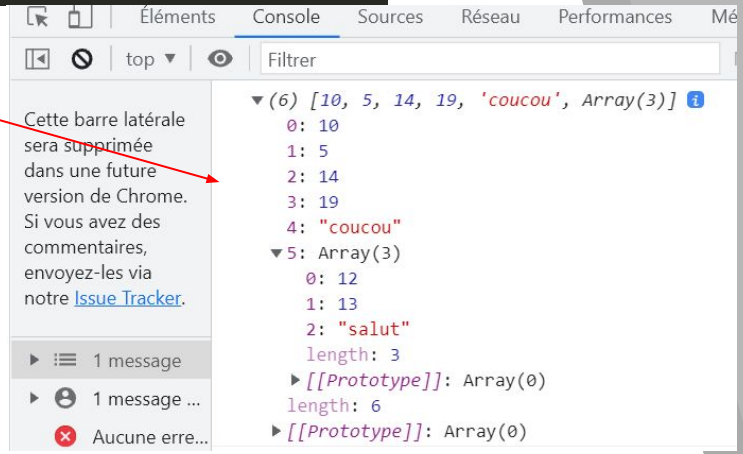
```
const notes = [10, 5, 14, 19];
console.log(notes);
```



D'ailleurs, avec un "console.log" de notre tableau, on peut voir les index. On dit tableau indicé, car il n'y a pas de colonne, juste l'index des lignes.

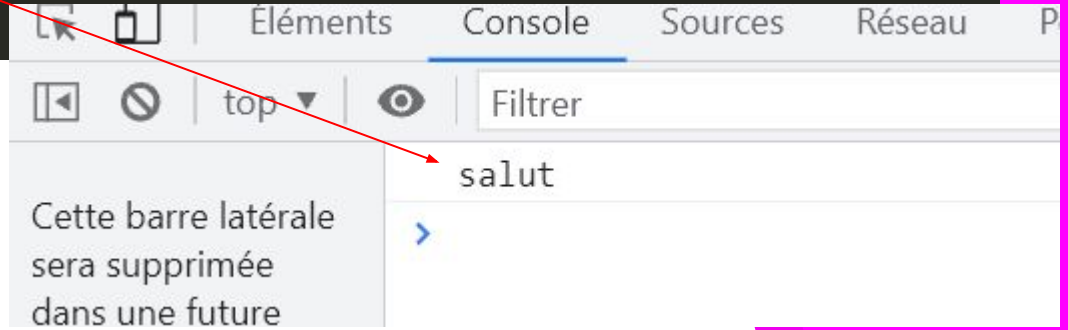
Algorithmie - Les Tableaux

```
const notes = [10, 5, 14, 19,'coucou',[12,13,'salut']];
console.log(notes);
```



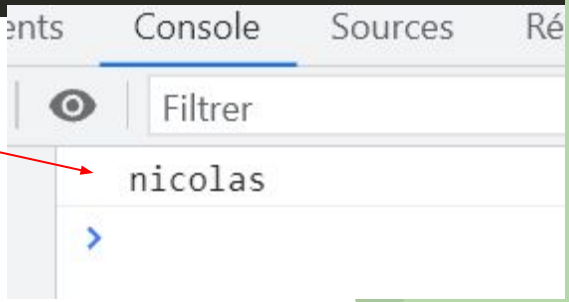
Dans un tableau indicé (ou autres), il peut y avoir des chiffres, mots et même d'autres tableau…A votre avis, comment je fais pour afficher "salut" ?

```
const notes = [10, 5, 14, 19,'coucou',[12,13,'salut']];
console.log(notes[5][2]);
```



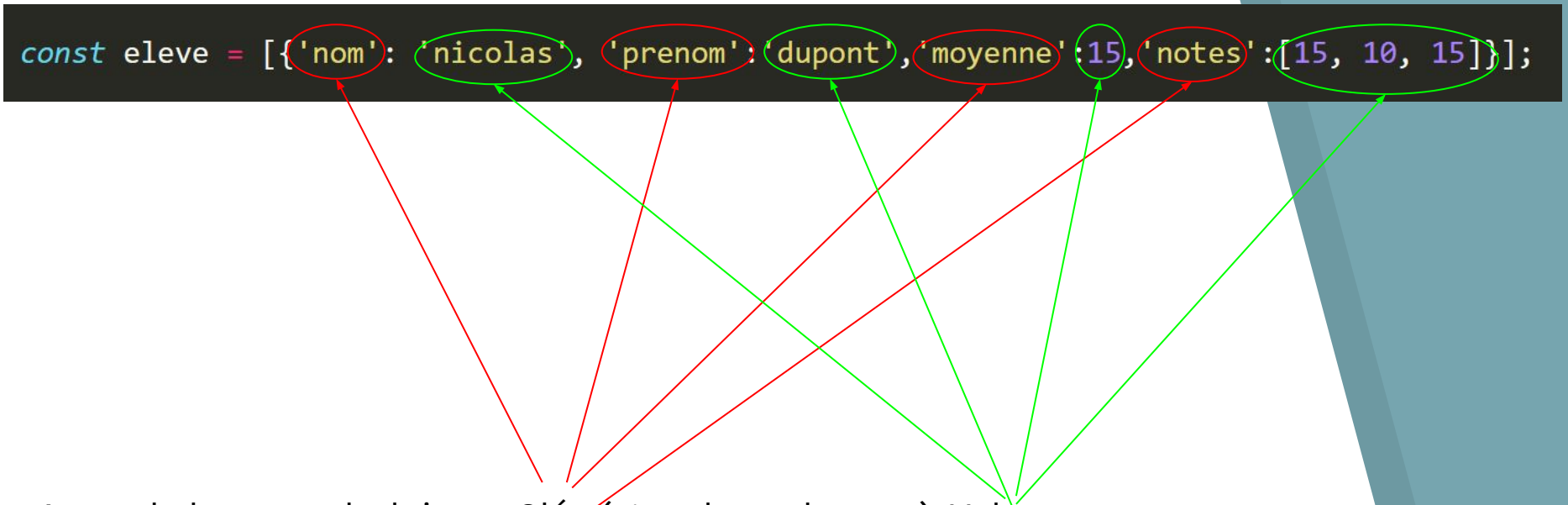
Pour sélectionner mon tableau qui est dans mon tableau “notes”, je dois sélectionner son index, qui est 5. Une fois sélectionné, je dois trouver l’index de “salut” dans ce tableau, qui est 3.

```
const eleve = [{'nom': 'nicolas', 'prenom':'dupont','moyenne':15,'notes':[15, 10, 15]}];
console.log(eleve[0].nom);
```



Voici un tableau associatif (avec des colonnes). Ce tableau n'a qu'une ligne. En JS, une ligne dans un tableau est entre accolades.
Pour sélectionner une valeur,il faut cette syntaxe :
tableau[numero_de_ligne].nom_de_la_colonne

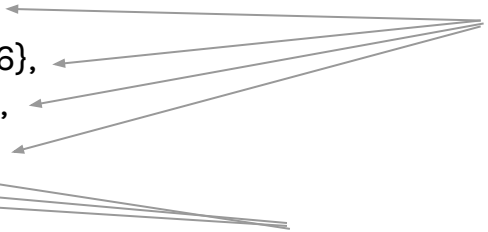
```
const eleve = [{'nom': 'nicolas', 'prenom':'dupont','moyenne':15,'notes':[15, 10, 15]}];
```



Ayons le bon vocabulaire..... Clés (c'est les colonnes), Valeurs

Algorithmie - Les Tableaux

```
const data = [
        {nom:'dupont', prenom: 'jean', age: 34},
        {nom:'pouley', prenom: 'martine', age: 56},
        {nom:'mary', prenom: 'thomas', age: 22},
        {nom:'dupuis', prenom: 'ines', age: 30}
];
```



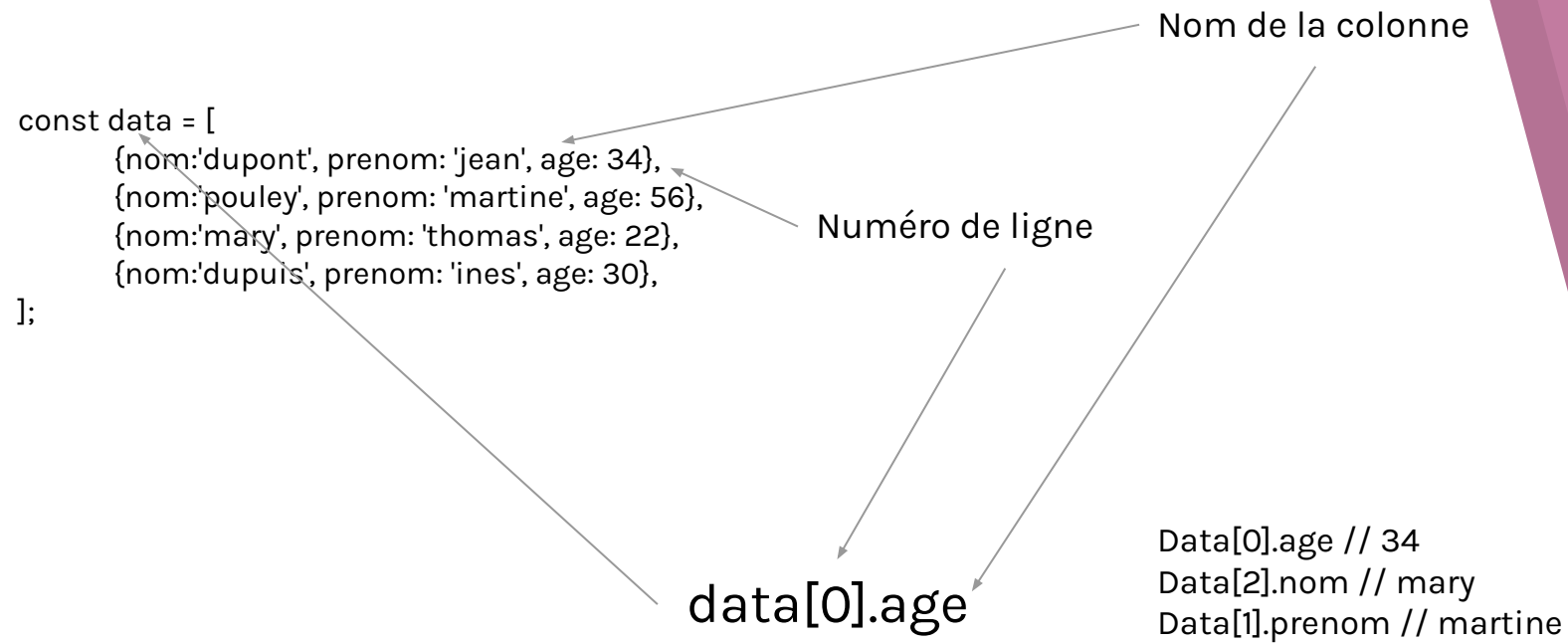
Lignes du tableau

Nom des colonnes (clés)

“Data” est un tableau associatif (avec des colonnes en fait)

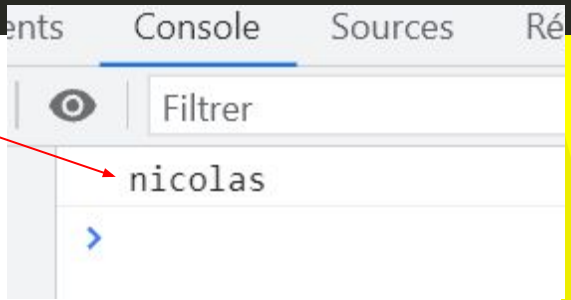
Autre exemple pour expliquer la structure d’un tableau associatif

Algorithmie - Les Tableaux



Autre exemple pour expliquer la syntaxe de sélection d'une valeur dans un tableau associatif

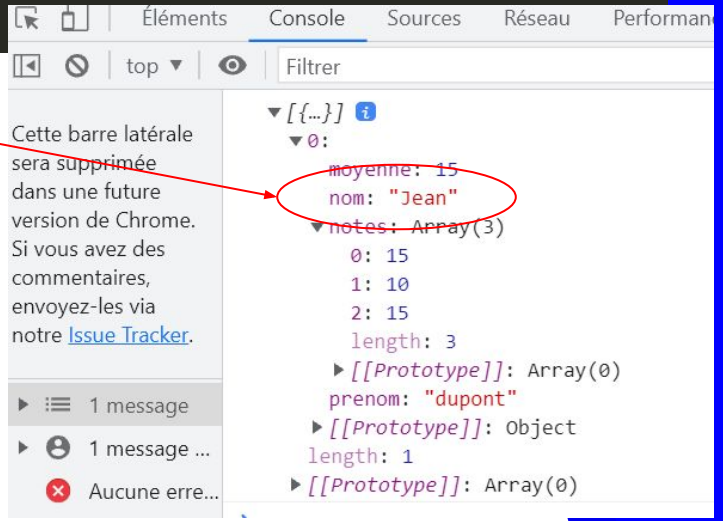
```
const eleve = [{'nom': 'nicolas', 'prenom':'dupont','moyenne':15,'notes':[15, 10, 15]}];
console.log(eleve[0]['nom']);
```



Cette syntaxe est aussi possible :
Tableau[numero_de_ligne][nom_de_la_colonne]
J'ai un faible pour l'autre syntaxe (avec le point)

Algorithmie - Les Tableaux

```
const eleve = [{'nom': 'nicolas', 'prenom':'dupont','moyenne':15,'notes':[15, 10, 15]}];
eleve[0].nom = 'Jean';
console.log(eleve);
```



Il est possible de modifier la valeur d'un tableau en assignant une nouvelle valeur à la colonne souhaitée.
Ci-dessus, j'assigne "Jean" à la clé (colonne) "nom"

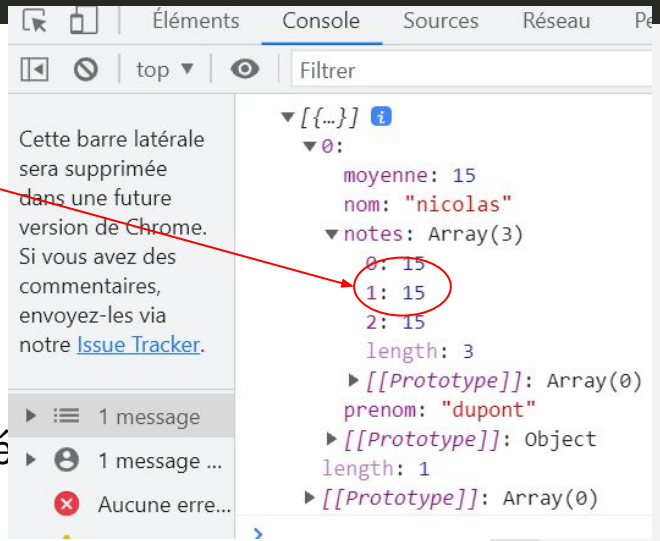
Algorithmie - Les Tableaux

```
const eleve = [{'nom': 'nicolas', 'prenom':'dupont','moyenne':15,'notes':[15, 10, 15]}];
```

Petit exercice, comment je fais pour modifier ma note de 10 en 15 ???

Algorithmie - Les Tableaux

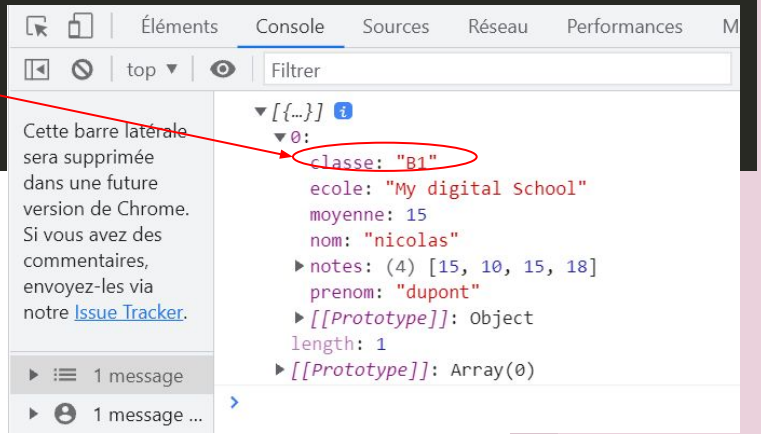
```
const eleve = [{'nom': 'nicolas', 'prenom':'dupont','moyenne':15,'notes':[15, 10, 15]}];
eleve[0].notes[1] = 15;
console.log(eleve);
```



1/ Je sélectionne la note eleve[0].notes[1] (tableau[numé
[index_de_la_note])
2/ Je lui assigne une nouvelle valeur : eleve[0].notes[1] = 15
3/ Voilà....

Algorithmie - Les Tableaux

```
const eleve = [{'nom': 'nicolas', 'prenom':'dupont','moyenne':15,'notes':[15, 10, 15]}];
eleve[0].classe = "B1";
eleve[0].ecole = "My digital School";
eleve[0].notes[3] = 18;
console.log(eleve);
```



Si je peux modifier des valeurs à la volé, je peux aussi ajouter de nouvelles clés….

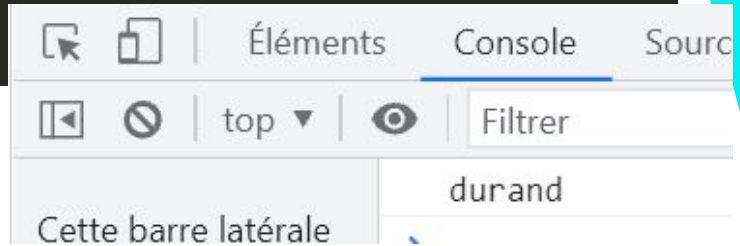
eleve[0].classe = "B1"; (tableau, ligne, nouvelle clé, valeur)

Algorithmie - Les Tableaux

```
const eleve = [
   {'nom': 'nicolas', 'prenom':'dupont','moyenne':15,'notes':[15, 10, 15]},
   {'nom': 'sophie', 'prenom':'durand','moyenne':18,'notes':[20, 18, 14]}
];

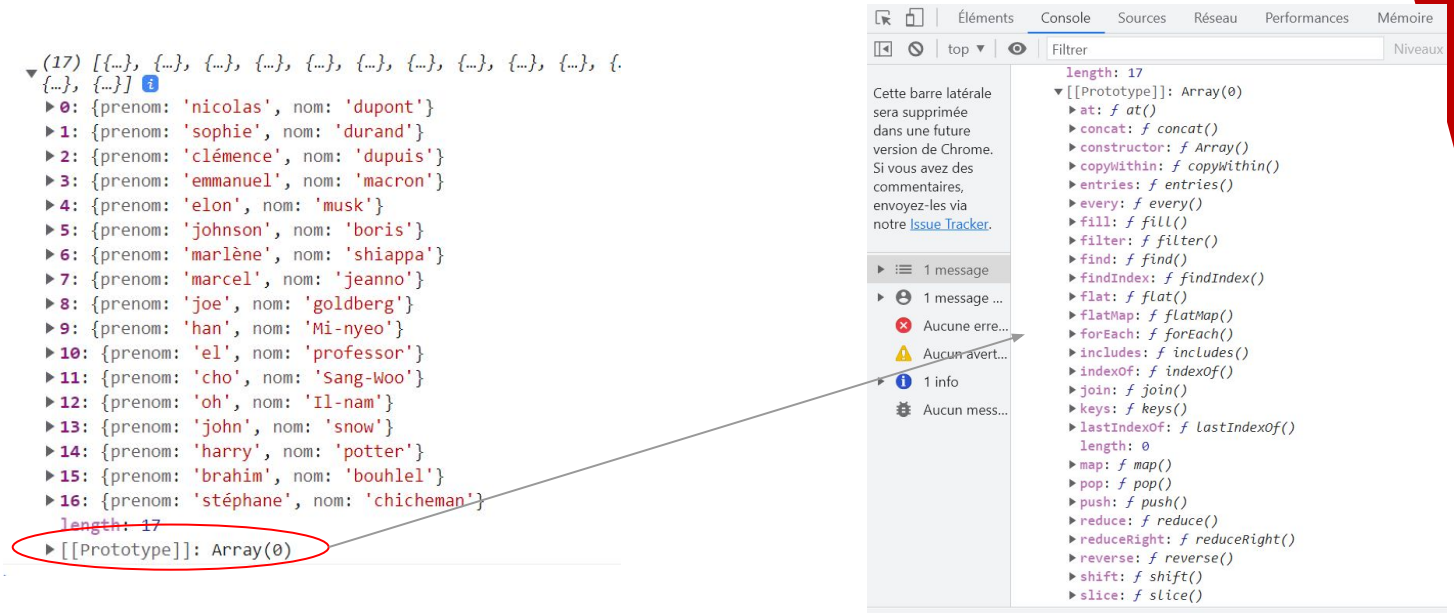


console.log(eleve[1].prenom);
```



Du coup, si j'ai 2 lignes (ou plus), j'ai juste à passer l'index de la ligne que je veux puis la clé pour afficher sa valeur.

Algorithmie - Les Tableaux



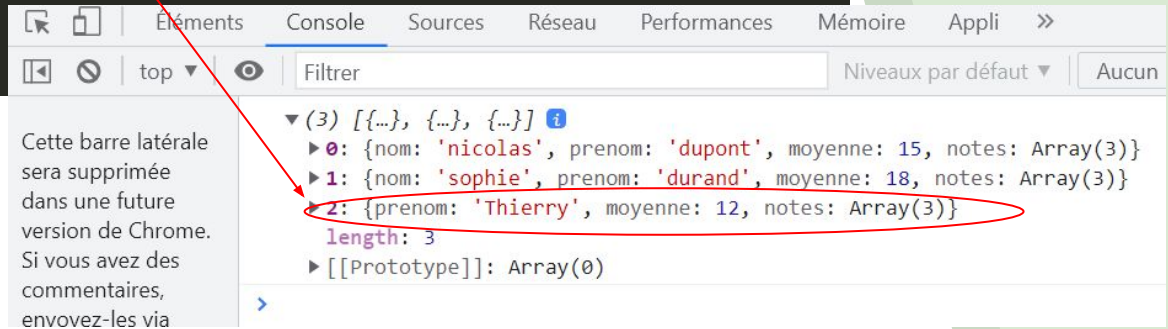
Lorsque je fais un console.log d'un tableau, en plus de m'afficher le détail de ce dernier, à la fin, on peut voir écrit dans la console "prototype"....JS essaierai-t-il de me faire passer un message ??

Et bien oui ! si on clique dessus, on voit que JS nous donne plusieurs méthodes (entre autres), pour manipuler les tableaux... Voyons en quelques unes

```
const eleve = [
   {'nom': 'nicolas', 'prenom':'dupont','moyenne':15,'notes':[15, 10, 15]},
  {'nom': 'sophie', 'prenom':'durand','moyenne':18,'notes':[20, 18, 14]}
];

eleve.push( {'nom': 'Henry', 'prenom':'Thierry','moyenne':12,'notes':[12, 12, 12]} );

console.log(eleve);
```



.push()

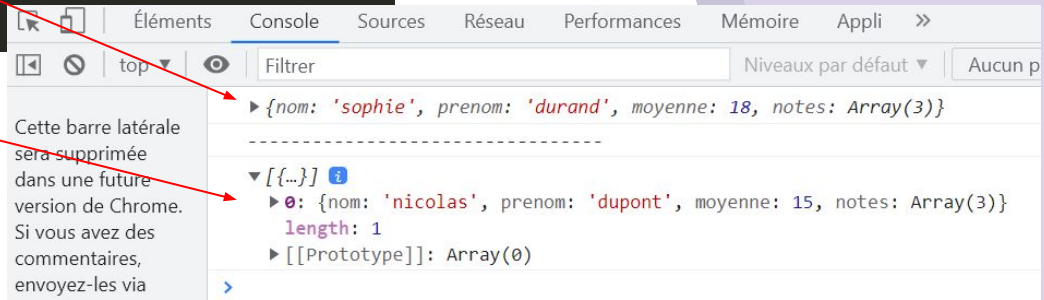
La méthode .push() va permettre d'ajouter une ligne **à la fin** d'un tableau et nous retourner la taille du tableau après ajout. Voici la syntaxe : tableau.push({clé : valeur}). Attention, dans le cas d'un tableau associatif, il faut autant de clé (colonne) que dans les lignes déjà présentes. Sinon vous aurez des problèmes plus tard.

Algorithmie - Les Tableaux

```
const eleve = [
   {'nom': 'nicolas', 'prenom':'dupont','moyenne':15,'notes':[15, 10, 15]},
  {'nom': 'sophie', 'prenom':'durand','moyenne':18,'notes':[20, 18, 14]}
];

let ligneSupprimee = eleve.pop();

console.log(ligneSupprimee);
console.log(`--------------------------------`);
console.log(eleve);
```



.pop()

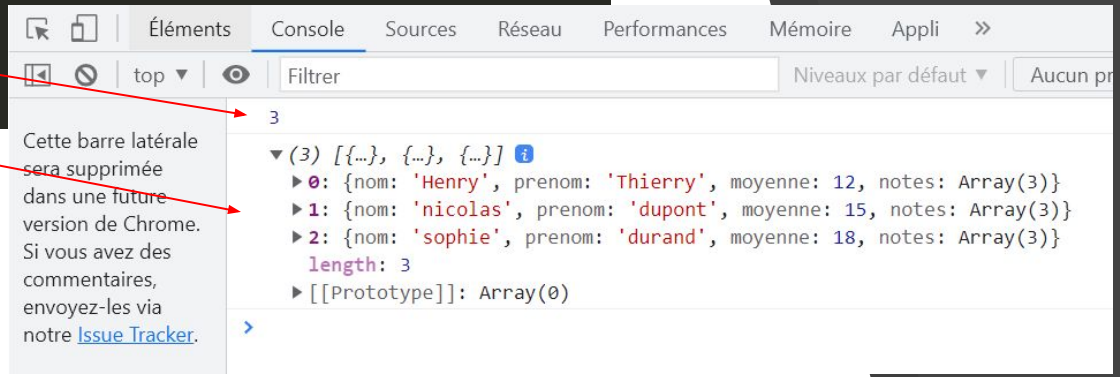
La méthode .pop() va supprimer le dernier élément du tableau et nous retourner l'élément qu'elle vient de supprimer (si souhaité). Dans l'exemple au-dessus, on constate que ma ligne supprimée est bien celle de Sophie et qu'il reste juste "Nicolas" dans mon tableau.
Syntaxe : tableau.pop()

```
const eleve = [
   {'nom': 'nicolas', 'prenom':'dupont','moyenne':15,'notes':[15, 10, 15]},
  {'nom': 'sophie', 'prenom':'durand','moyenne':18,'notes':[20, 18, 14]}
];


let tailleDuTableau = eleve.unshift({'nom': 'Henry', 'prenom':'Thierry','moyenne':12,'notes':[12, 12, 12
   ]});

console.log(tailleDuTableau);

console.log(eleve);
```



.unshift()

La méthode .unshift() à la même fonction que .push() sauf qu'elle va ajouter une ligne (ou élément) au début du tableau et nous retourner la nouvelle taille du tableau après l'ajout.
Syntaxe : tableau.unshift({clé : valeur})

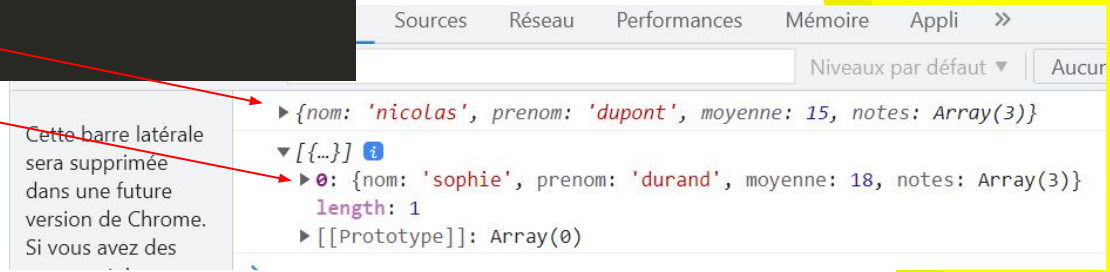
Algorithmie - Les Tableaux

```
const eleve = [
   {'nom': 'nicolas', 'prenom':'dupont','moyenne':15,'notes':[15, 10, 15]},
  {'nom': 'sophie', 'prenom':'durand','moyenne':18,'notes':[20, 18, 14]}
];


let ligneSupprimee = eleve.shift();

console.log(ligneSupprimee);

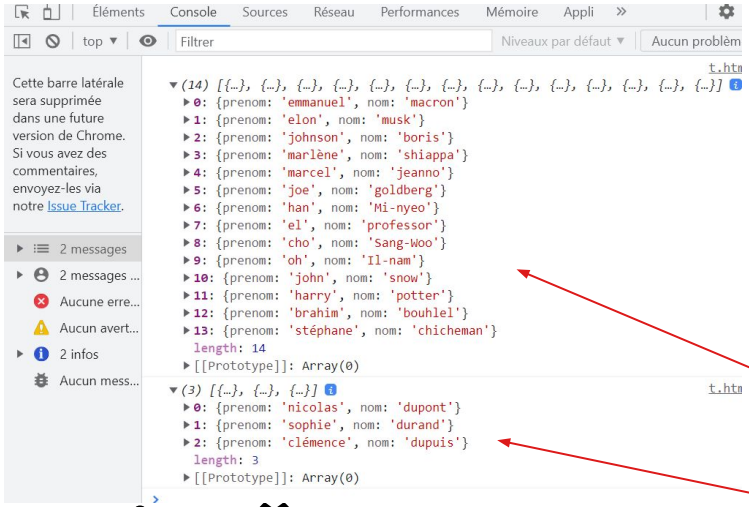
console.log(eleve);
```



.shift()

La méthode .shift() à la même fonction que .pop(), sauf qu'elle va supprimer la première ligne (ou élément) d'un tableau et nous retourner la ligne (ou l'élément) supprimé).
Syntaxe : tableau.shift()

Algorithmie - Les Tableaux



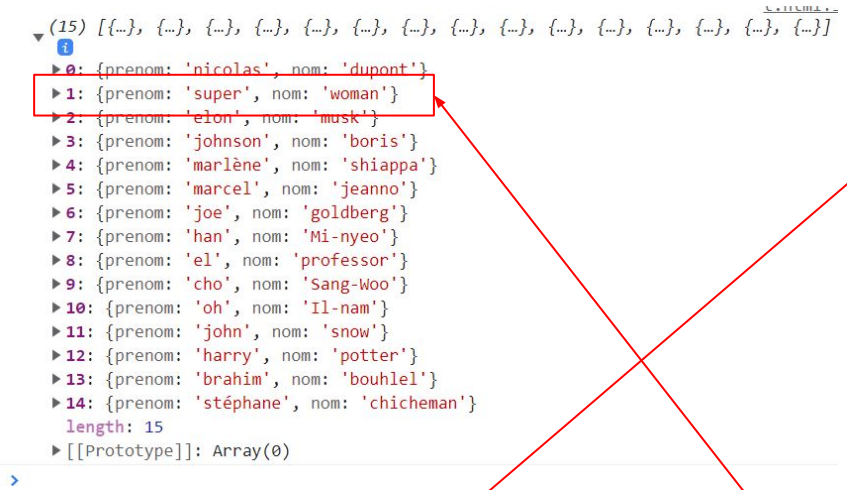

```
const eleves = [
    {'prenom': 'nicolas', 'nom':'dupont'},
    {'prenom': 'sophie', 'nom':'durand'},
    {'prenom': 'clémence', 'nom':'dupuis'},
    {'prenom': 'emmanuel', 'nom':'macron'},
    {'prenom': 'elon', 'nom':'musk'},
    {'prenom': 'johnson', 'nom':'boris'},
    {'prenom': 'marlène', 'nom':'shiappa'},
    {'prenom': 'marcel', 'nom':'jeanno'},
    {'prenom': 'joe', 'nom':'goldberg'},
    {'prenom': 'han', 'nom':'Mi-nyeo'},
    {'prenom': 'el', 'nom':'professor'},
    {'prenom': 'cho', 'nom':'Sang-Woo'},
    {'prenom': 'oh', 'nom':'Il-nam'},
    {'prenom': 'john', 'nom':'snow'},
    {'prenom': 'harry', 'nom':'potter'},
    {'prenom': 'brahim', 'nom':'bouhlel'},
    {'prenom': 'stéphane', 'nom':'chicheman'},
];


let elementsSupprimes = eleves.splice(3);
console.log(elementsSupprimes);
console.log(eleves);
```

.splice()

La méthode .splice() peut modifier, remplacer, ajouter des éléments n'importe où dans le tableau.
Cette méthode va également retourner un tableau contenant les éléments supprimés.
Syntaxe : tableau.splice(indexDeDepart, nbDeSuppression, elementAajouter)

Algorithmie - Les Tableaux




```
const eleves = [
    {'prenom': 'nicolas', 'nom':'dupont'},
  {'prenom': 'sophie', 'nom':'durand'},
  {'prenom': 'clémence', 'nom':'dupuis'},
  {'prenom': 'emmanuel', 'nom':'macron'},
  {'prenom': 'elon', 'nom':'musk'},
  {'prenom': 'johnson', 'nom':'boris'},
  {'prenom': 'marlène', 'nom':'shiappa'},
  {'prenom': 'marcel', 'nom':'jeanno'},
  {'prenom': 'joe', 'nom':'goldberg'},
  {'prenom': 'han', 'nom':'Mi-nyeo'},
  {'prenom': 'el', 'nom':'professor'},
  {'prenom': 'cho', 'nom':'Sang-Woo'},
  {'prenom': 'oh', 'nom':'Il-nam'},
  {'prenom': 'john', 'nom':'snow'},
  {'prenom': 'harry', 'nom':'potter'},
  {'prenom': 'brahim', 'nom':'bouhlel'},
  {'prenom': 'stéphane', 'nom':'chicheman'},
];


eleves.splice(1,3,{'prenom': 'super', 'nom':'woman'});

console.log(eleves);
```

.splice()

Exemple :

eleves.splice(1,3,{'prenom': 'super', 'nom':'woman'});

Je lui demande “Dès la deuxième ligne (1), tu vas m’en supprimer 3 et tu vas m’ajouter une ligne ({'prenom': 'super', 'nom':'woman'})

Algorithmie - Les Tableaux

```
(18) [{…}, {…}, {…}, {…}, {…}, {…}, {…}, {…}, {…}, {…}, {…}, {…}, {…}, {…}, {…}, {…}, {…}, {…}]
  0: {prenom: 'nicolas', nom: 'dupont'}
  1: {prenom: 'sophie', nom: 'durand'}
  2: {prenom: 'super', nom: 'woman'}
  3: {prenom: 'clémence', nom: 'dupuis'}
  4: {prenom: 'emmanuel', nom: 'macron'}
  5: {prenom: 'elon', nom: 'musk'}
  6: {prenom: 'johnson', nom: 'boris'}
  7: {prenom: 'marlène', nom: 'shiappa'}
  8: {prenom: 'marcel', nom: 'jeanno'}
  9: {prenom: 'joe', nom: 'goldberg'}
  10: {prenom: 'han', nom: 'Mi-nyeo'}
  11: {prenom: 'el', nom: 'professor'}
  12: {prenom: 'cho', nom: 'Sang-Woo'}
  13: {prenom: 'oh', nom: 'Il-nam'}
  14: {prenom: 'john', nom: 'snow'}
  15: {prenom: 'harry', nom: 'potter'}
  16: {prenom: 'brahim', nom: 'bouhlel'}
  17: {prenom: 'stéphane', nom: 'chicheman'}
  length: 18
  [[Prototype]]: Array(0)
```

```
const eleves = [
   {'prenom': 'nicolas', 'nom':'dupont'},
  {'prenom': 'sophie', 'nom':'durand'},
  {'prenom': 'clémence', 'nom':'dupuis'},
  {'prenom': 'emmanuel', 'nom':'macron'},
  {'prenom': 'elon', 'nom':'musk'},
  {'prenom': 'johnson', 'nom':'boris'},
  {'prenom': 'marlène', 'nom':'shiappa'},
  {'prenom': 'marcel', 'nom':'jeanno'},
  {'prenom': 'joe', 'nom':'goldberg'},
  {'prenom': 'han', 'nom':'Mi-nyeo'},
  {'prenom': 'el', 'nom':'professor'},
  {'prenom': 'cho', 'nom':'Sang-Woo'},
  {'prenom': 'oh', 'nom':'Il-nam'},
  {'prenom': 'john', 'nom':'snow'},
  {'prenom': 'harry', 'nom':'potter'},
  {'prenom': 'brahim', 'nom':'bouhlel'},
  {'prenom': 'stéphane', 'nom':'chicheman'},
];


eleves.splice(2,0,{'prenom': 'super', 'nom':'woman'});

console.log(eleves);
```

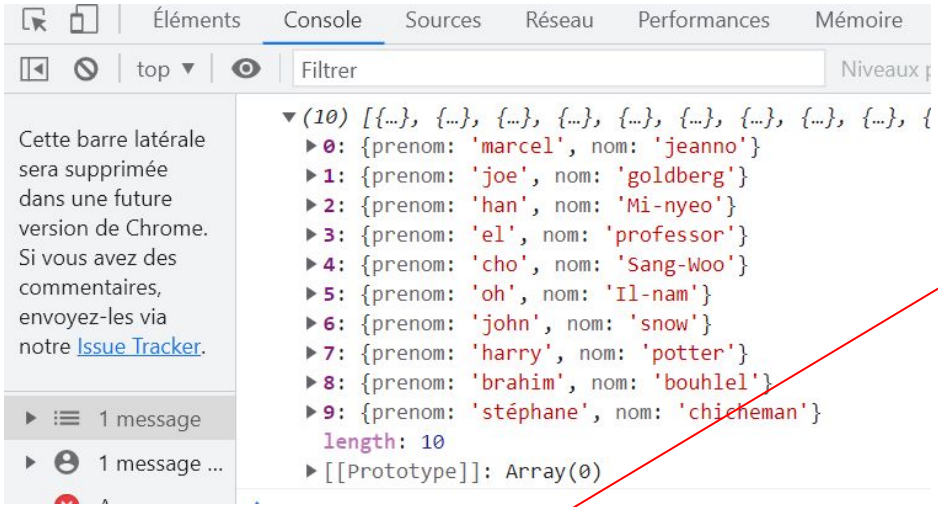
.splice()

Exemple :

eleves.splice(2,0,{'prenom': 'super', 'nom':'woman'});

Je lui demande “Dès la troisième ligne (2), tu ne vas pas m’en supprimer (0) et tu vas m’ajouter une ligne ({'prenom': 'super', 'nom':'woman'})

Algorithmie - Les Tableaux




```
const eleves = [
   {'prenom': 'nicolas', 'nom':'dupont'},
  {'prenom': 'sophie', 'nom':'durand'},
  {'prenom': 'clémence', 'nom':'dupuis'},
  {'prenom': 'emmanuel', 'nom':'macron'},
  {'prenom': 'elon', 'nom':'musk'},
  {'prenom': 'johnson', 'nom':'boris'},
  {'prenom': 'marlène', 'nom':'shiappa'},
  {'prenom': 'marcel', 'nom':'jeanno'},
  {'prenom': 'joe', 'nom':'goldberg'},
  {'prenom': 'han', 'nom':'Mi-nyeo'},
  {'prenom': 'el', 'nom':'professor'},
  {'prenom': 'cho', 'nom':'Sang-Woo'},
  {'prenom': 'oh', 'nom':'Il-nam'},
  {'prenom': 'john', 'nom':'snow'},
  {'prenom': 'harry', 'nom':'potter'},
  {'prenom': 'brahim', 'nom':'bouhlel'},
  {'prenom': 'stéphane', 'nom':'chicheman'},
];


eleves.splice(0,7);

console.log(eleves);
```

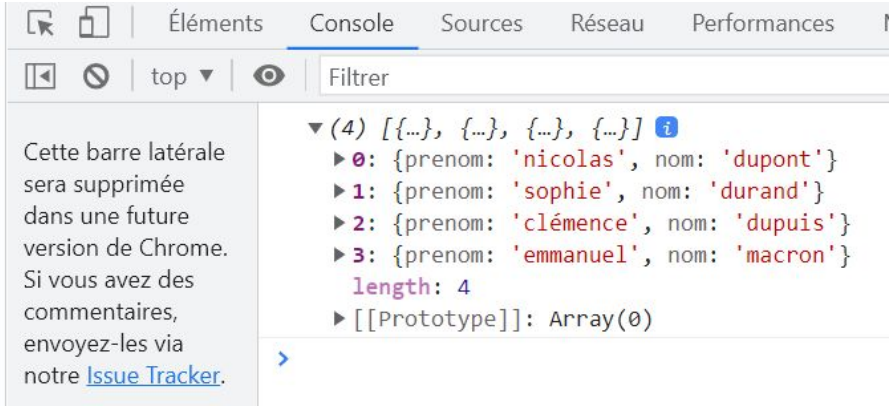
.splice()

Exemple :

eleves.splice(0,7);

Je lui demande “Dès la première ligne (0), tu vas m’en supprimer 7 et tu vas rien m’ajouter.

Algorithmie - Les Tableaux




```
const eleves = [
   {'prenom': 'nicolas', 'nom':'dupont'},
  {'prenom': 'sophie', 'nom':'durand'},
  {'prenom': 'clémence', 'nom':'dupuis'},
  {'prenom': 'emmanuel', 'nom':'macron'},
  {'prenom': 'elon', 'nom':'musk'},
  {'prenom': 'johnson', 'nom':'boris'},
  {'prenom': 'marlène', 'nom':'shiappa'},
  {'prenom': 'marcel', 'nom':'jeanno'},
  {'prenom': 'joe', 'nom':'goldberg'},
  {'prenom': 'han', 'nom':'Mi-nyeo'},
  {'prenom': 'el', 'nom':'professor'},
  {'prenom': 'cho', 'nom':'Sang-Woo'},
  {'prenom': 'oh', 'nom':'Il-nam'},
  {'prenom': 'john', 'nom':'snow'},
  {'prenom': 'harry', 'nom':'potter'},
  {'prenom': 'brahim', 'nom':'bouhlel'},
  {'prenom': 'stéphane', 'nom':'chicheman'},
];


eleves.splice(4);

console.log(eleves);
```

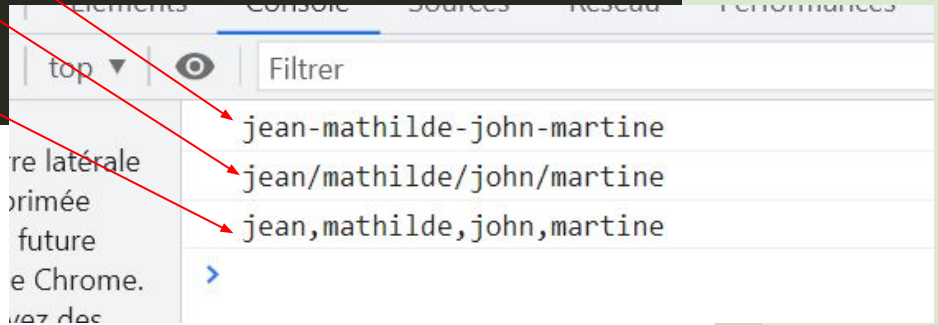
.splice()

Exemple :

eleves.splice(4);

Je lui demande “Dès la cinquième ligne (5), Tu me supprimes tout !

```
const noms = ['jean', 'mathilde', 'john', 'martine'];
console.log(noms.join('-'));
console.log(noms.join('/'));
console.log(noms.join());
```



.join()

La méthode join() retourne une chaîne de caractères créée en concaténant les différentes valeurs d'un tableau. Le séparateur utilisé par défaut sera la virgule mais nous allons également pouvoir passer le séparateur de notre choix en **argument** de join().
Syntaxe : tableau.join(séparateurSouhaité);

```
const eleves = [
   {'prenom': 'nicolas', 'nom':'dupont'},
  {'prenom': 'sophie', 'nom':'durand'},
  {'prenom': 'clémence', 'nom':'dupuis'},
  {'prenom': 'emmanuel', 'nom':'macron'},
  {'prenom': 'elon', 'nom':'musk'},
  {'prenom': 'johnson', 'nom':'boris'},
  {'prenom': 'marlène', 'nom':'shiappa'},
  {'prenom': 'marcel', 'nom':'jeanno'},
  {'prenom': 'joe', 'nom':'goldberg'},
  {'prenom': 'han', 'nom':'Mi-nyeo'},
  {'prenom': 'el', 'nom':'professor'},
  {'prenom': 'cho', 'nom':'Sang-Woo'},
  {'prenom': 'oh', 'nom':'Il-nam'},
  {'prenom': 'john', 'nom':'snow'},
  {'prenom': 'harry', 'nom':'potter'},
  {'prenom': 'brahim', 'nom':'bouhlel'},
  {'prenom': 'stéphane', 'nom':'chicheman'},
];


const noms = ['jean', 'mathilde', 'john', 'martine'];

console.log(noms.concat(eleves));
```

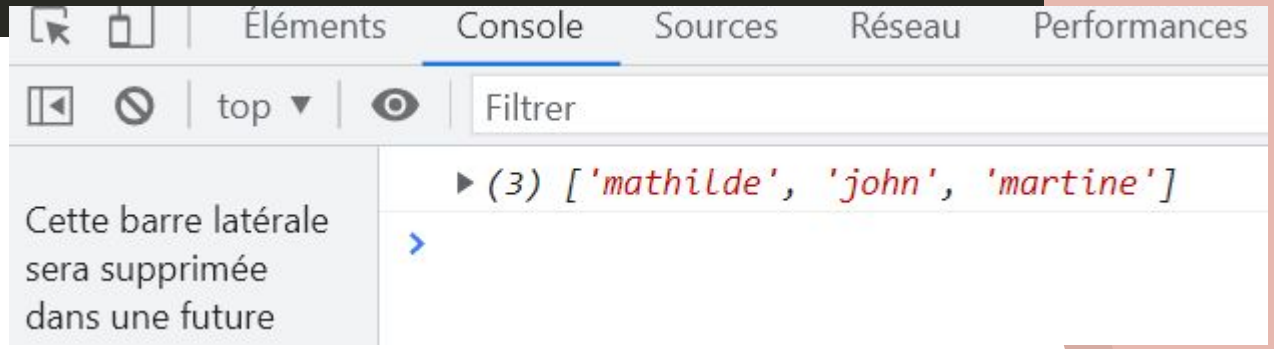
```
(21) ['jean', 'mathilde', 'john', 'martine', {…}, {…},
{…}, {…}, {…}, {…}, {…}, {…}, {…}, {…}, {…}, {…}, {…}]
  0: "jean"
  1: "mathilde"
  2: "john"
  3: "martine"
  4: {prenom: 'nicolas', nom: 'dupont'}
  5: {prenom: 'sophie', nom: 'durand'}
  6: {prenom: 'clémence', nom: 'dupuis'}
  7: {prenom: 'emmanuel', nom: 'macron'}
  8: {prenom: 'elon', nom: 'musk'}
  9: {prenom: 'johnson', nom: 'boris'}
  10: {prenom: 'marlène', nom: 'shiappa'}
  11: {prenom: 'marcel', nom: 'jeanno'}
  12: {prenom: 'joe', nom: 'goldberg'}
  13: {prenom: 'han', nom: 'Mi-nyeo'}
  14: {prenom: 'el', nom: 'professor'}
  15: {prenom: 'cho', nom: 'Sang-Woo'}
  16: {prenom: 'oh', nom: 'Il-nam'}
  17: {prenom: 'john', nom: 'snow'}
  18: {prenom: 'harry', nom: 'potter'}
  19: {prenom: 'brahim', nom: 'bouhlel'}
  20: {prenom: 'stéphane', nom: 'chicheman'}
  length: 21
  [[Prototype]]: Array(0)
```

.concat()

La méthode concat() va nous permettre de fusionner différents tableaux entre eux pour en créer un nouveau qu'elle va renvoyer.
On peut fusionner plusieurs tableaux à la fois…
Syntaxe : tableau.concat(tableauAfusionner)

```
const noms = ['jean', 'mathilde', 'john', 'martine','kevin','julie'];

console.log(noms.slice(1,4));
```



.slice()

La méthode slice() va extraire des éléments d'un tableau sous forme…de tableau.
Syntaxe : tableau.slice(indexDeDepart, indexDeFin)
Dans l'exemple ci-dessus, on voit que j'extrais à partir du deuxième nom (inclus) jusqu'au cinquième (exclus).

```
const noms = ['jean', 'mathilde', 'john', 'martine','kevin','julie'];

console.log(noms.slice(1));
```

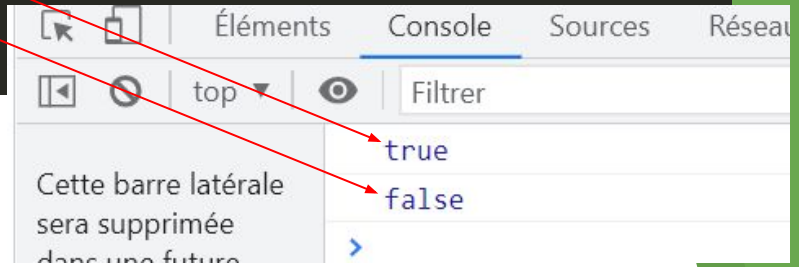


.slice()

Si je ne passe qu'un argument (index de départ) et pas le deuxième (index de fin), la méthode va extraire tout à partir de l'index de départ.

```
const noms = ['jean', 'mathilde', 'john', 'martine','kevin','julie'];

console.log(noms.includes('john'));
console.log(noms.includes('julien'));
```

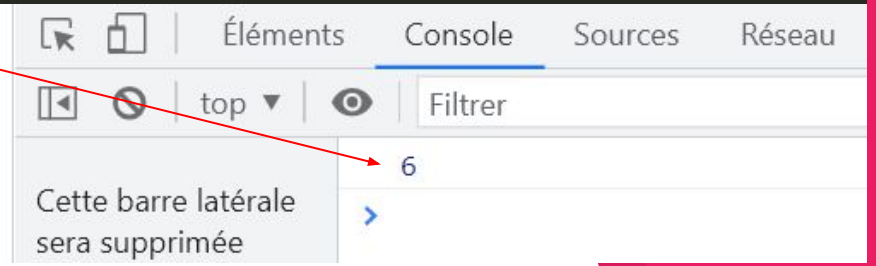


.includes()

La méthode .includes() sert à vérifier si un élément en particulier est présent dans le tableau… Il retourne un booléen (true si il est dans le tableau, false si il n'y est pas)
Syntaxe : tableau.includes('ceQueVousCherchezDansLeTableau');
La méthode "indexOf()" est similaire….

```
const noms = ['jean', 'mathilde', 'john', 'martine','kevin','julie'];

console.log(noms.length);
```



.length

.length n'est pas une méthode mais une propriété. Elle retourne la taille d'un tableau (son nombre de ligne)
Syntaxe : tableau.length

Algorithmie, Les Boucles

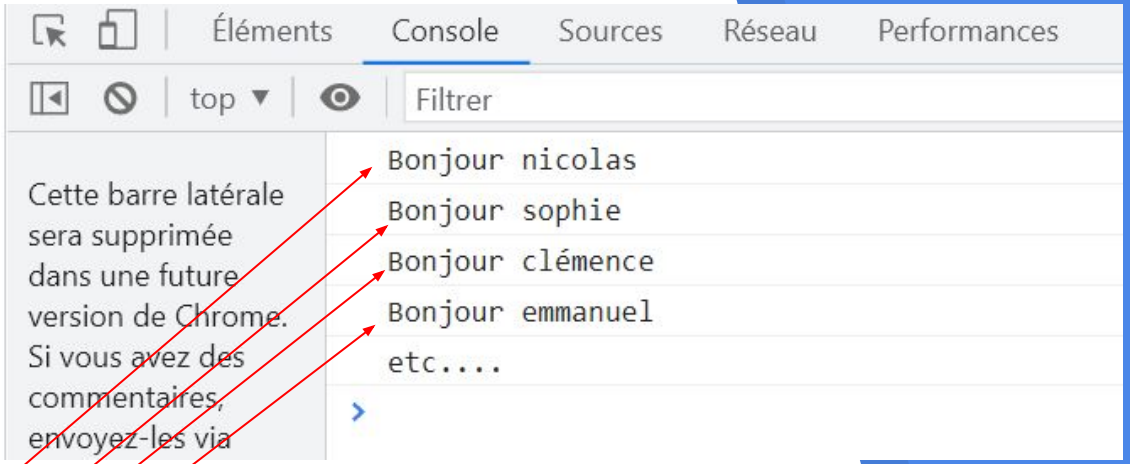
Les boucles vont nous permettre d'exécuter plusieurs fois un bloc de code ou parcourir des tableaux, c'est-à-dire d'exécuter un code « en boucle » tant qu'une condition donnée est vérifiée et donc ainsi nous faire gagner beaucoup de temps dans l'écriture de nos scripts.

Lorsqu'on code, on va en effet souvent devoir exécuter plusieurs fois un même code. Utiliser une boucle nous permet de n'écrire le code qu'on doit exécuter plusieurs fois qu'une seule fois.

Algorithmie - Les Boucles

```
const eleves = [
   {'prenom': 'nicolas', 'nom':'dupont'},
  {'prenom': 'sophie', 'nom':'durand'},
  {'prenom': 'clémence', 'nom':'dupuis'},
  {'prenom': 'emmanuel', 'nom':'macron'},
  {'prenom': 'elon', 'nom':'musk'},
  {'prenom': 'johnson', 'nom':'boris'},
  {'prenom': 'marlène', 'nom':'shiappa'},
  {'prenom': 'marcel', 'nom':'jeanno'},
  {'prenom': 'joe', 'nom':'goldberg'},
  {'prenom': 'han', 'nom':'Mi-nyeo'},
  {'prenom': 'el', 'nom':'professor'},
  {'prenom': 'cho', 'nom':'Sang-Woo'},
  {'prenom': 'oh', 'nom':'Il-nam'},
  {'prenom': 'john', 'nom':'snow'},
  {'prenom': 'harry', 'nom':'potter'},
  {'prenom': 'brahim', 'nom':'bouhlel'},
  {'prenom': 'stéphane', 'nom':'chicheman'},
];

console.log(`Bonjour ${eleves[0].prenom}`);
console.log(`Bonjour ${eleves[1].prenom}`);
console.log(`Bonjour ${eleves[2].prenom}`);
console.log(`Bonjour ${eleves[3].prenom}`);
console.log(`etc....`);
```

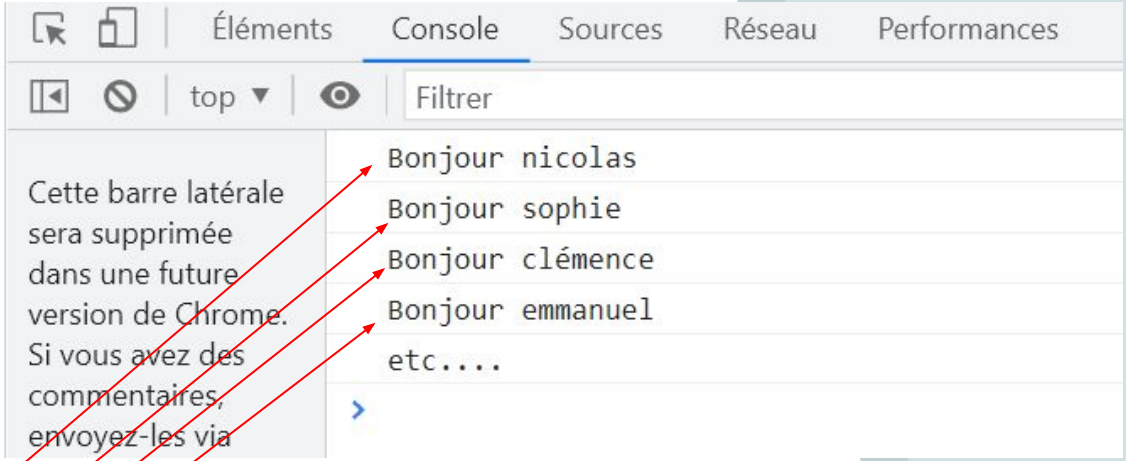


Imaginons que nous souhaitons saluer tous les élèves.... Faire un console.log() et les appeler un par un serait long....

Algorithmie - Les Boucles

```
const eleves = [
   {'prenom': 'nicolas', 'nom':'dupont'},
  {'prenom': 'sophie', 'nom':'durand'},
  {'prenom': 'clémence', 'nom':'dupuis'},
  {'prenom': 'emmanuel', 'nom':'macron'},
  {'prenom': 'elon', 'nom':'musk'},
  {'prenom': 'johnson', 'nom':'boris'},
  {'prenom': 'marlène', 'nom':'shiappa'},
  {'prenom': 'marcel', 'nom':'jeanno'},
  {'prenom': 'joe', 'nom':'goldberg'},
  {'prenom': 'han', 'nom':'Mi-nyeo'},
  {'prenom': 'el', 'nom':'professor'},
  {'prenom': 'cho', 'nom':'Sang-Woo'},
  {'prenom': 'oh', 'nom':'Il-nam'},
  {'prenom': 'john', 'nom':'snow'},
  {'prenom': 'harry', 'nom':'potter'},
  {'prenom': 'brahim', 'nom':'bouhlel'},
  {'prenom': 'stéphane', 'nom':'chicheman'},
];

console.log(`Bonjour ${eleves[0].prenom}`);
console.log(`Bonjour ${eleves[1].prenom}`);
console.log(`Bonjour ${eleves[2].prenom}`);
console.log(`Bonjour ${eleves[3].prenom}`);
console.log(`etc....`);
```

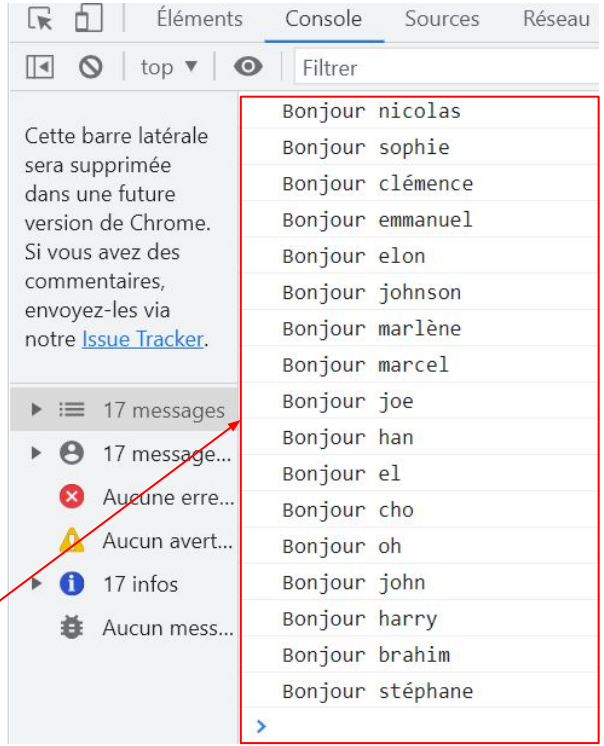


Et en admettant qu'on les fasse tous, si une personne ajoute ou supprime des élèves, nous devrons rajouter ou supprimer manuellement (donc toujours vérifier le tableau).

Algorithmie - Les Boucles

```
const eleves = [
   {'prenom': 'nicolas', 'nom':'dupont'},
  {'prenom': 'sophie', 'nom':'durand'},
  {'prenom': 'clémence', 'nom':'dupuis'},
  {'prenom': 'emmanuel', 'nom':'macron'},
  {'prenom': 'elon', 'nom':'musk'},
  {'prenom': 'johnson', 'nom':'boris'},
  {'prenom': 'marlène', 'nom':'shiappa'},
  {'prenom': 'marcel', 'nom':'jeanno'},
  {'prenom': 'joe', 'nom':'goldberg'},
  {'prenom': 'han', 'nom':'Mi-nyeo'},
  {'prenom': 'el', 'nom':'professor'},
  {'prenom': 'cho', 'nom':'Sang-Woo'},
  {'prenom': 'oh', 'nom':'Il-nam'},
  {'prenom': 'john', 'nom':'snow'},
  {'prenom': 'harry', 'nom':'potter'},
  {'prenom': 'brahim', 'nom':'bouhlel'},
  {'prenom': 'stéphane', 'nom':'chicheman'},
];

for (var i = 0; i < eleves.length; i++) {
   console.log(`Bonjour ${eleves[i].prenom}`);
}
```



Avec la boucle, en 3 lignes, c'est réglé. Tous mes élèves seront salués et si une personnes modifie le tableau, la liste de salutations se mettra à jour automatiquement...

Algorithmie - Les Boucles

```
for (var i = 0; i < Things.length; i++) {
   Things[i]
}


array.forEach( function(element, index) {
   // statements
});


while (condition) {
   // statement
}
```

Il existe plusieurs types de boucle... Nous allons en voir 3 for, forEach et while

Algorithmie - Les Boucles

```
while (tant_que_je_suis_vrai) {
    console.log('je serais exécuté....')
}
```

La signification d'une boucle while est très simple. JS exécute l'instruction tant que l'expression de la boucle while est évaluée comme true. La valeur de l'expression est vérifiée à chaque début de boucle, et, si la valeur change durant l'exécution de l'instruction, l'exécution ne s'arrêtera qu'à la fin de l'itération (chaque fois que JS exécute l'instruction, on appelle cela une itération). Si l'expression du while est false avant la première itération, l'instruction ne sera jamais exécutée.

Algorithmie - Les Boucles

```
const eleves = [
   {'prenom': 'nicolas', 'nom':'dupont'},
  {'prenom': 'sophie', 'nom':'durand'},
  {'prenom': 'clémence', 'nom':'dupuis'},
  {'prenom': 'emmanuel', 'nom':'macron'},
  {'prenom': 'elon', 'nom':'musk'},
  {'prenom': 'johnson', 'nom':'boris'},
  {'prenom': 'marlène', 'nom':'shiappa'},
  {'prenom': 'marcel', 'nom':'jeanno'},
  {'prenom': 'joe', 'nom':'goldberg'},
  {'prenom': 'han', 'nom':'Mi-nyeo'},
  {'prenom': 'el', 'nom':'professor'},
  {'prenom': 'cho', 'nom':'Sang-Woo'},
  {'prenom': 'oh', 'nom':'Il-nam'},
  {'prenom': 'john', 'nom':'snow'},
  {'prenom': 'harry', 'nom':'potter'},
  {'prenom': 'brahim', 'nom':'bouhlel'},
  {'prenom': 'stéphane', 'nom':'chicheman'},
];

let i = 0;
while(i < eleves.length) {
   console.log(`Bonjour ${eleves[i].prenom }`);
   i++;
}
```

Pouvez-vous m'expliquez svp ?

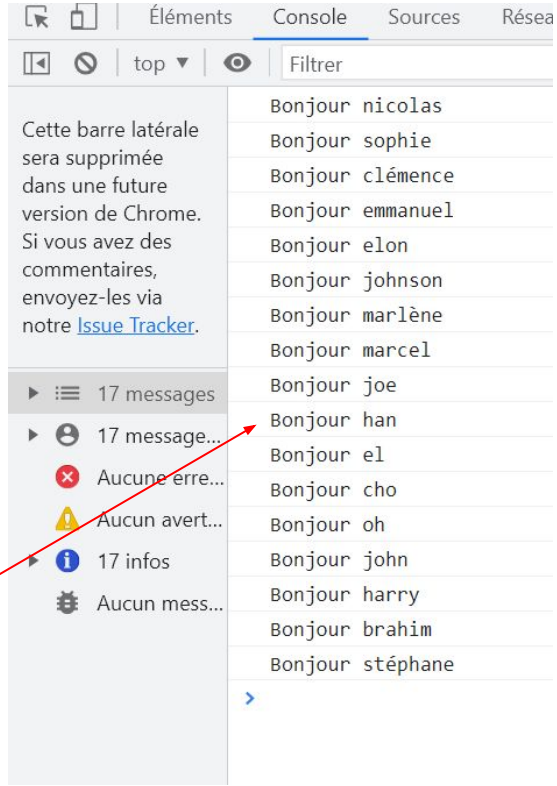
Algorithmie - Les Boucles

```
//On créé une variable i (comme index) et
//on l'initialise à 0....
let i = 0;
//On dit "tant que i est strictement plus petit que la taille du tableau 'élèves'"
//La boucle s'arrêtera forcément à la fin du tableau car eleves.length = 17
//Et comme l'index de la dernière ligne est "16" (on commence à 0)
while(i < eleves.length) {
    //On affiche "Bonjour nom_de_l'eleve"
    //Donc au premier tour (i vaut 0) eleves[0].prenom soit 'nicolas'
    //Donc au deuxième tour (i vaut 1) eleves[1].prenom soit 'sophie'
    //...
    console.log(`Bonjour ${eleves[i].prenom }`);
    //On incrémente i de 1 (donc à la fin du premier tour, i sera égal à 1)
    //On incrémente i de 1 (donc à la fin du deuxième tour, i sera égal à 2)
    //....
    //i ne dépassera jamais 16 car dans ma condition il doit s'arrêter quand i
    //n'est plus strictement inférieur à 17 (donc 16)
    i++;
}
```

Algorithmie - Les Boucles

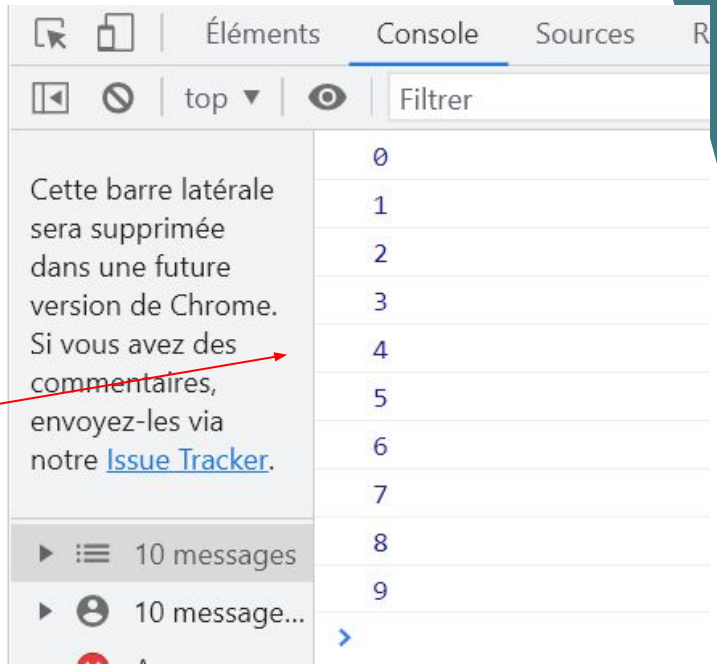
```
const eleves = [
   {'prenom': 'nicolas', 'nom':'dupont'},
  {'prenom': 'sophie', 'nom':'durand'},
  {'prenom': 'clémence', 'nom':'dupuis'},
  {'prenom': 'emmanuel', 'nom':'macron'},
  {'prenom': 'elon', 'nom':'musk'},
  {'prenom': 'johnson', 'nom':'boris'},
  {'prenom': 'marlène', 'nom':'shiappa'},
  {'prenom': 'marcel', 'nom':'jeanno'},
  {'prenom': 'joe', 'nom':'goldberg'},
  {'prenom': 'han', 'nom':'Mi-nyeo'},
  {'prenom': 'el', 'nom':'professor'},
  {'prenom': 'cho', 'nom':'Sang-Woo'},
  {'prenom': 'oh', 'nom':'Il-nam'},
  {'prenom': 'john', 'nom':'snow'},
  {'prenom': 'harry', 'nom':'potter'},
  {'prenom': 'brahim', 'nom':'bouhlel'},
  {'prenom': 'stéphane', 'nom':'chicheman'},
];


let i = 0;
while(i < eleves.length) {
   console.log(`Bonjour ${eleves[i].prenom }`);
   i++;
}
```



Algorithmie - Les Boucles

```
let i = 0;
while(i < 10) {
    console.log(i);
    i++;
}
```



La boucle while n'a pas besoin de tableau pour fonctionner, juste une limite...

Algorithmie - Les Boucles

```
for (var i = 0; i < limite_a_pas_depasser; i++) {
    console.log('je m\'affiche...')
}
```

La boucle for...

La première expression (expr1) est évaluée (exécutée), quoi qu'il arrive au début de la boucle.

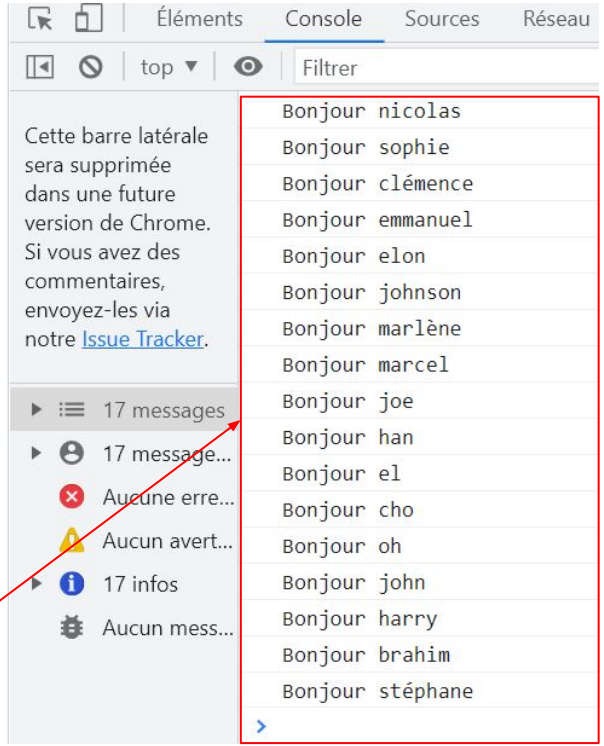
Au début de chaque itération, l'expression expr2 est évaluée. Si l'évaluation vaut true, la boucle continue et les commandes sont exécutées. Si l'évaluation vaut false, l'exécution de la boucle s'arrête.

À la fin de chaque itération, l'expression expr3 est évaluée (exécutée).

Algorithmie - Les Boucles

```
const eleves = [
   {'prenom': 'nicolas', 'nom':'dupont'},
  {'prenom': 'sophie', 'nom':'durand'},
  {'prenom': 'clémence', 'nom':'dupuis'},
  {'prenom': 'emmanuel', 'nom':'macron'},
  {'prenom': 'elon', 'nom':'musk'},
  {'prenom': 'johnson', 'nom':'boris'},
  {'prenom': 'marlène', 'nom':'shiappa'},
  {'prenom': 'marcel', 'nom':'jeanno'},
  {'prenom': 'joe', 'nom':'goldberg'},
  {'prenom': 'han', 'nom':'Mi-nyeo'},
  {'prenom': 'el', 'nom':'professor'},
  {'prenom': 'cho', 'nom':'Sang-Woo'},
  {'prenom': 'oh', 'nom':'Il-nam'},
  {'prenom': 'john', 'nom':'snow'},
  {'prenom': 'harry', 'nom':'potter'},
  {'prenom': 'brahim', 'nom':'bouhlel'},
  {'prenom': 'stéphane', 'nom':'chicheman'},
];

for (var i = 0; i < eleves.length; i++) {
   console.log(`Bonjour ${eleves[i].prenom}`);
}
```



Algorithmie - Les Boucles

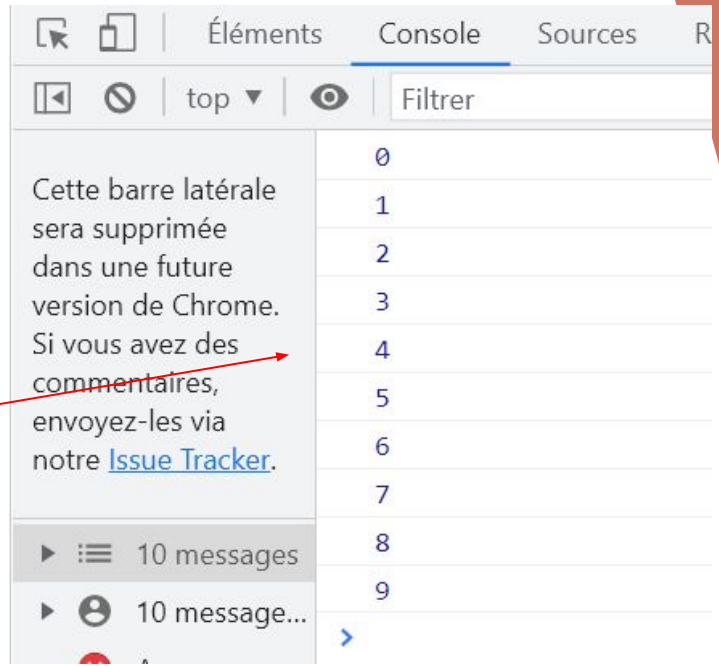
```
let i = 0;
while(i < 10) {
    console.log(i);
    i++;
}
```

```
for (var i = 0; i < 10; i++) {
    console.log(i);
}
```

La boucle “for” c’est juste une boucle while mieux organisée car l’index de départ, la condition et l’incrémentation (ou décrémentation) sont intégrés dans la boucles.

Algorithmie - Les Boucles

```
for (var i = 0; i < 10; i++) {
    console.log(i);
}
```



Comme la boucle “while”, la boucle “for” n’a pas besoin de tableau pour fonctionner, juste une limite…

Algorithmie - Les Boucles

```
array.forEach( function(element, index) {
    // statements
});
```

Ca, c'est une boucle "forEach" (en français "pour chaque"). Contrairement au 2 autres boucles, celle-ci à besoin d'un tableau pour être exécutée.
La boucle "forEach" n'a pas besoin de limite, elle va parcourir toutes les lignes d'un tableau de la première à la dernière et s'arrêter d'elle-même.
Pas besoin non plus de lui indiquer un index de départ, c'est 0 (la première ligne) et pas besoin non plus de lui dire d'incrémenter ou de décrémenter, elle incrémente.

Algorithmie - Les Boucles

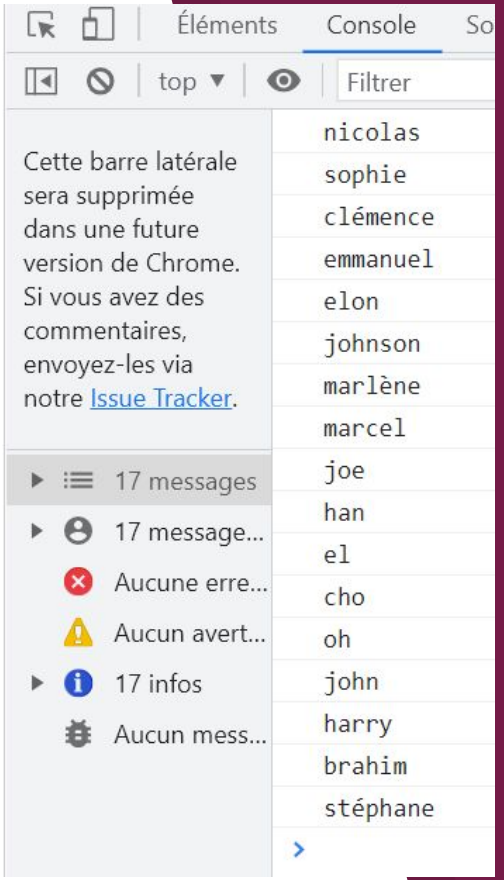
```
const eleves = [
   {'prenom': 'nicolas', 'nom':'dupont'},
   {'prenom': 'sophie', 'nom':'durand'},
   {'prenom': 'clémence', 'nom':'dupuis'},
   {'prenom': 'emmanuel', 'nom':'macron'},
   {'prenom': 'elon', 'nom':'musk'},
   {'prenom': 'johnson', 'nom':'boris'},
   {'prenom': 'marlène', 'nom':'shiappa'},
   {'prenom': 'marcel', 'nom':'jeanno'},
   {'prenom': 'joe', 'nom':'goldberg'},
   {'prenom': 'han', 'nom':'Mi-nyeo'},
   {'prenom': 'el', 'nom':'professor'},
   {'prenom': 'cho', 'nom':'Sang-Woo'},
   {'prenom': 'oh', 'nom':'Il-nam'},
   {'prenom': 'john', 'nom':'snow'},
   {'prenom': 'harry', 'nom':'potter'},
   {'prenom': 'brahim', 'nom':'bouhlel'},
   {'prenom': 'stéphane', 'nom':'chicheman'},
];


eleves.forEach( function(element, index) {
   console.log(element.prenom); // eleves[index].prenom
});
```

Ligne du tableau en cours (équivalent de eleves[i]

Tableau à parcourir

Index de la ligne

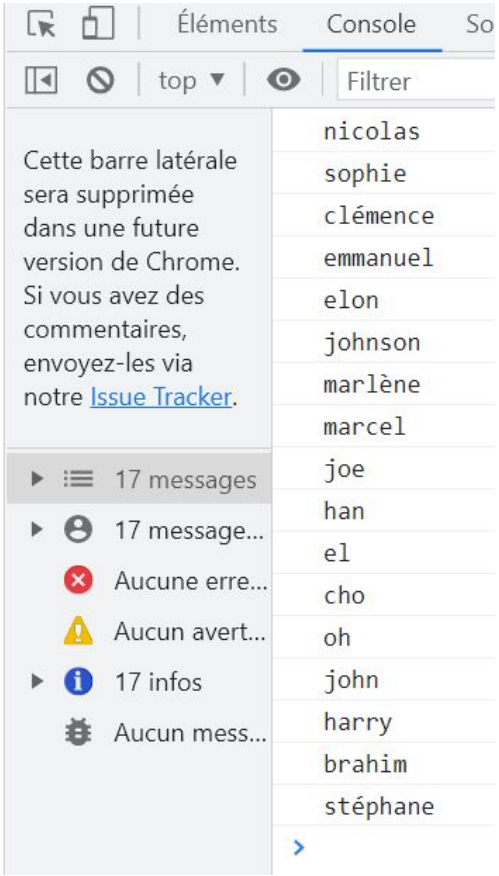


Du coup pour sélectionner une valeur, il suffit de mettre le nom de la clé à la suite d'"element" (qui est la ligne en cours)...

Algorithmie - Les Boucles

```
const eleves = [
   {'prenom': 'nicolas', 'nom':'dupont'},
   {'prenom': 'sophie', 'nom':'durand'},
   {'prenom': 'clémence', 'nom':'dupuis'},
   {'prenom': 'emmanuel', 'nom':'macron'},
   {'prenom': 'elon', 'nom':'musk'},
   {'prenom': 'johnson', 'nom':'boris'},
   {'prenom': 'marlène', 'nom':'shiappa'},
   {'prenom': 'marcel', 'nom':'jeanno'},
   {'prenom': 'joe', 'nom':'goldberg'},
   {'prenom': 'han', 'nom':'Mi-nyeo'},
   {'prenom': 'el', 'nom':'professor'},
   {'prenom': 'cho', 'nom':'Sang-Woo'},
   {'prenom': 'oh', 'nom':'Il-nam'},
   {'prenom': 'john', 'nom':'snow'},
   {'prenom': 'harry', 'nom':'potter'},
   {'prenom': 'brahim', 'nom':'bouhlel'},
   {'prenom': 'stéphane', 'nom':'chicheman'},
];


eleves.forEach( function(element, index) {
   console.log(eleves[index].prenom);
});
```



Du coup, cette façon fonctionne aussi pour récupérer les valeurs...

Algorithmie - Les Boucles

```
eleves.forEach( function(element) {
    console.log(element.prenom);
});
```

Juste pour info, l'argument "index" est optionnel.

Algorithmie, Les Fonctions

Une fonction correspond à un bloc de code nommé et réutilisable et dont le but est d'effectuer une tâche précise. Elles sont très utilisées en JS et dans les autres langages pouvant les supporter.

Elles servent à éviter la duplication de code et avoir une meilleure organisation. Elles peuvent retourner une valeur (chiffre, texte, booléen, tableau,...) ou ne rien retourner et juste exécuter un code.

Ils existent 2 type de fonctions, les natives et les prédéfinies. Dans les 2 cas, elles sont aussi appelées"méthodes"

```
eleves.unshift();
eleves.pop();


console.log('coucou');
eleves.push({'prenom': 'stéphane', 'nom':'chicheman'});
eleves.splice(0,2);
```

Sans nous en rendre compte, nous avons utilisé plusieurs fonctions natives de JS. push(), pop(), shift(), log()…. Une fonction se reconnaît à sa syntaxe particulière : **nomDeLaFonction()**
Avec ses parenthèses (vides ou non), la fonction est facilement reconnaissable. Dans les parenthèses, on peut passer un ou plusieurs paramètres (arguments)…. Ils sont séparés par des virgules.

```
console.log('coucou');
eleves.push({'prenom': 'stéphane', 'nom':'chicheman'});
eleves.splice(0,2);
```

Les arguments peuvent être une chaîne de caractère, des nombres, un tableau, objet,.... Ou même une autre fonction.

Un argument peut être optionnel ou obligatoire

Algorithmie - Les Fonctions

```
eleves.forEach( function(element) {
    console.log(element.prenom);
});
```

Une fonction peut être nommée (ex : push()) ou non..... Dans le cas où elle ne l'est pas on dit qu'elle est anonyme.

La méthode (fonction) forEach à en arguments une fonction anonyme.

```
eleves.forEach( function(element) {
    console.log(element.prenom);
});
```

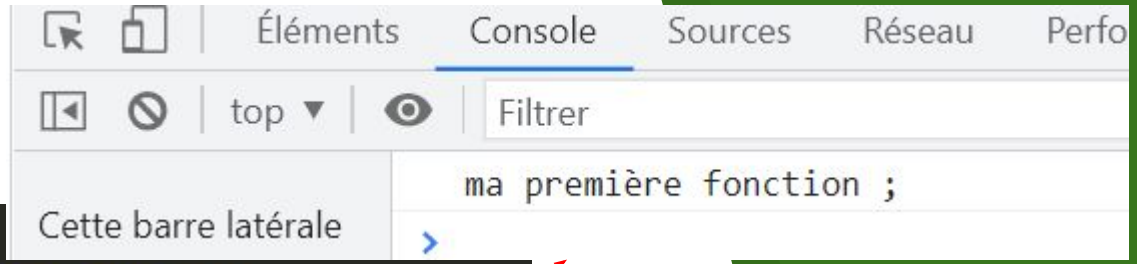
Quand nous utilisons une fonction sans que nous l'ayons prédéfinie (créé nous-même), on dit qu'elle est native. C'est à dire que JS l'a créé et mis à notre disposition. Nous n'avons plus qu'à l'appeler pour l'utiliser.

Algorithmie - Les Fonctions

```
function mon_nom_de_fonction(){
    //Mon code ici
}
```

Si nous voulons créer notre propre fonction, il faut l'initialiser (un peu comme une variable) avec la commande (mot) 'function', ensuite il faut lui donner un nom (ce sont les même règles de nommage que les variable, mettre 2 parenthèses (même si il n'y a pas d'arguments (paramètres) et 2 accolades avec entre deux le code à exécuter quand la fonction est appelée.

Algorithmie - Les Fonctions




```
function afficherDansConsole(){
    console.log('ma première fonction ;');
}


afficherDansConsole();
```

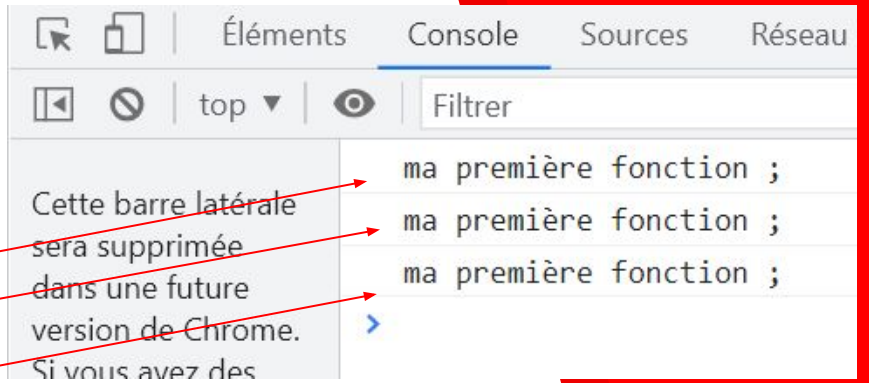
Nous venons de créer notre première fonction (sommaire certe). Cette dernière va juste afficher 'ma première fonction ;' dans la console (pas foufou mais faut un début un tout).

Une fois créée, j'appelle ma fonction juste en copiant le nom de la fonction…

Algorithmie - Les Fonctions

```
function afficherDansConsole(){
    console.log('ma première fonction ;');
}


afficherDansConsole();
afficherDansConsole();
afficherDansConsole();
```

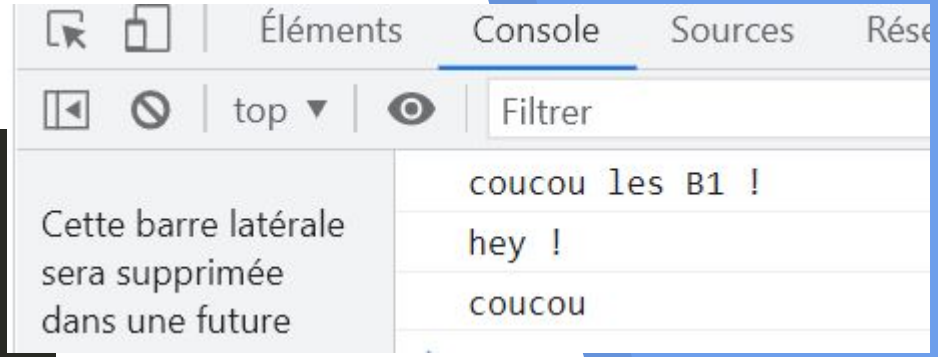


Bon, le truc c'est que je peux appeler ma fonction autant de fois que je veux, J'ai toujours le même texte.....y a pas trop d'intérêt.

Algorithmie - Les Fonctions

```
function afficherDansConsole($texte){
    console.log($texte);
}


afficherDansConsole('coucou les B1 !');
afficherDansConsole('hey !');
afficherDansConsole('coucou');
```

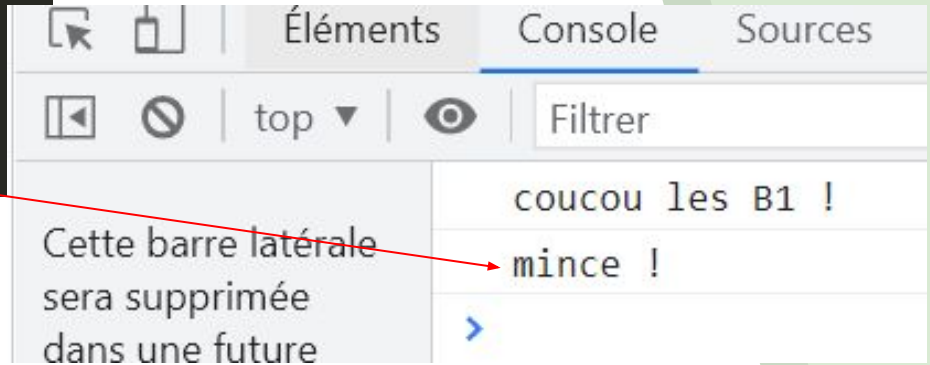


Pour changer un peu, je décide d'ajouter un argument que j'ai appelé '$texte'. "$texte" est une variable qui va s'ajouter dans mon console.log() et donc varier en fonction du contenu....

Algorithmie - Les Fonctions

```
function afficherDansConsole($texte='mince !'){
    console.log($texte);
}


afficherDansConsole('coucou les B1 !');
afficherDansConsole();
```

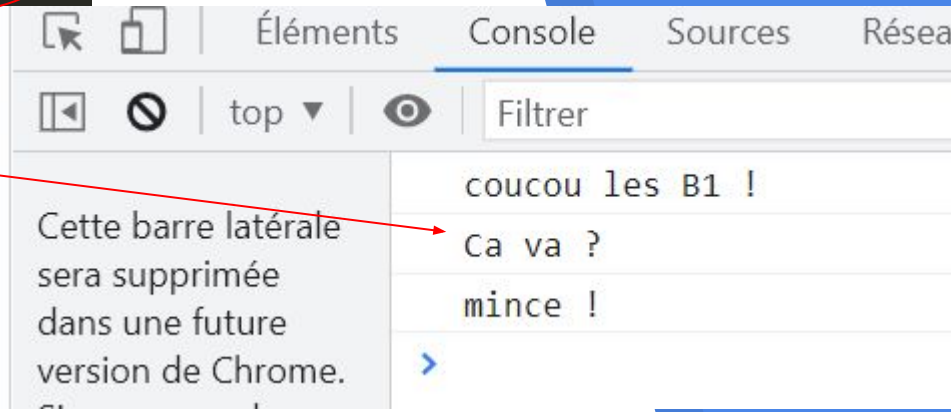


Je peux assigner une valeur par défaut à mon argument qui sera affiché si j'appelle ma fonction sans lui préciser de texte. En gros, on lui dit "si vide, affiche moi la valeur par défaut (qui est "mince !")"

Algorithmie - Les Fonctions

```
function afficherDansConsole($texte='mince !', $option=false){
    console.log($texte);
    if($option) console.log($option);
}


afficherDansConsole('coucou les B1 !', 'Ca va ?');
afficherDansConsole();
```



Nous venons de créer un second paramètre optionnel…
Avec une condition dans la fonction qui dit “si $option existe, affiche le, sinon rien”.

Algorithmie - Les Fonctions

```
//fonction déclarative
function somme(a, b){
    return a + b;
}

//fonction d'expression
let somme = function(a,b){
    return a + b;
}

// fonction fléchée
let somme = (a,b)=> a+b;

// fonction fléchée
let double = a => a*2;
```

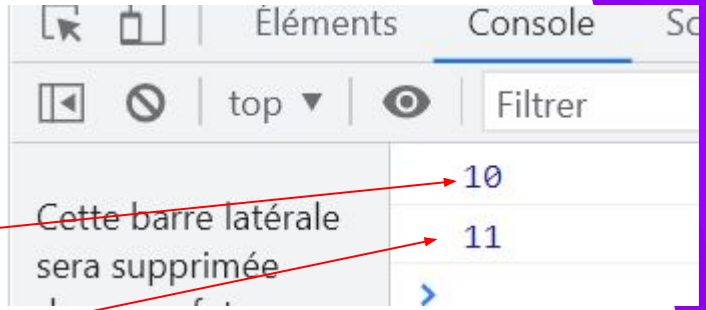
Autre syntaxe que vous pouvez croiser....

Algorithmie - Les Fonctions

```
//fonction déclarative
function somme(a, b){
    return a + b;
    console.log('je ne serais jamais exécuté');
}


console.log(somme(2,8));


let sum = somme(2,8);
sum++;
console.log(sum);
```



L'instruction “return” met fin à l'exécution d'une fonction et définit une valeur à renvoyer à la fonction appelante.
Une fonction sans “return” ne retourne rien, elle exécute un code.
L’avantage du “return” c’est qu’il est possible de stocker le résultat d’une fonction dans une variable soit pour l’afficher plus tard soit la manipuler puis l’afficher.

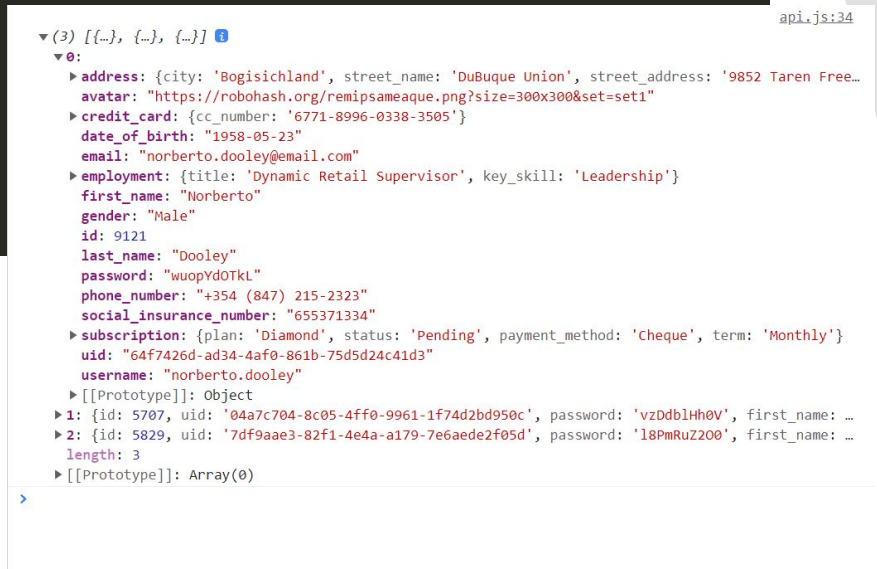
Algorithmie, AJAX

“L’AJAX” ou plutôt “l’Ajax” (Asynchronous JavaScript and XML) est aujourd’hui un terme générique utilisé pour désigner toute technique côté client (côté navigateur) permettant d’envoyer et de récupérer des données depuis un serveur et de mettre à jour dynamiquement le DOM sans nécessiter l’actualisation complète de la page.

En gros, il est possible de récupérer (ou d’envoyer) des données sur un serveur et de les importer directement dans notre page pour les traiter.

Algorithmie - AJAX

```
fetch('https://random-data-api.com/api/users/random_user?size=3')
.then((data)=>{
    return data.json();
})
.then((data)=>{
    console.log(data);
})
```



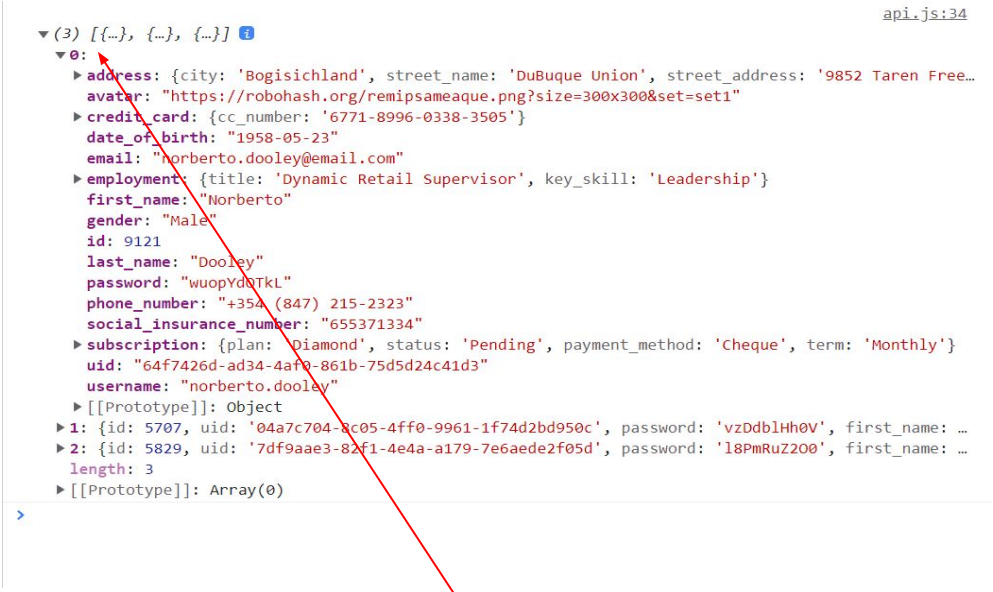
Dans cette exemple, j'utilise la fonction "fetch" (qui elle utilise des promesses, mais on y va doucement) qui va me récupérer des données via l'api "https://random-data-api.com/" (c'est une api de test).

Algorithmie - AJAX

```
fetch('https://random-data-api.com/api/users/random_user?size=3')
.then((data)=>{
    return data.json();
})
.then((data)=>{
    console.log(data);
})
```

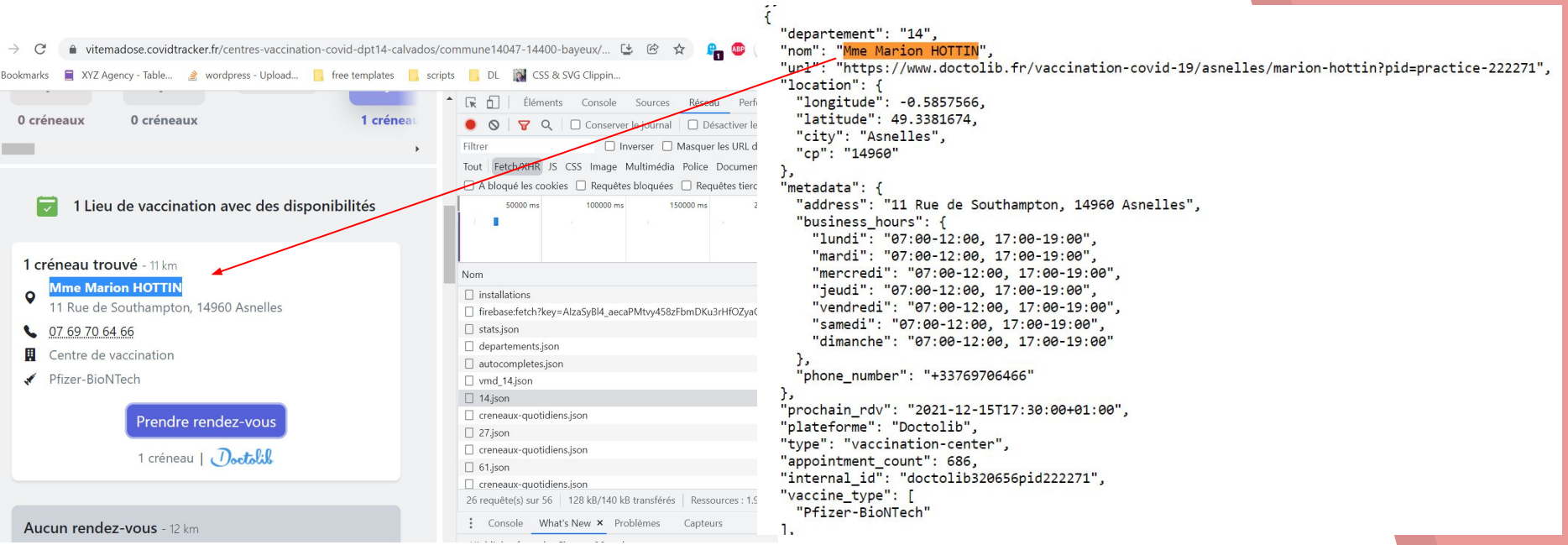
Dans un premier temps, fetch envoie une requête à un serveur : https://random-data-api.com/api/users/random_user
Dans un second temps, dès qu'il reçoit les données de ce serveur, il les converti on JSON (data.json())
Et enfin, dès que c'est fini, il les affiche....

Algorithmie - AJAX



La réponse de cette api est un tableau à 3 lignes. Chaque ligne correspond à Une personne avec des données personnelles (nom, genre, poste, email, adresse,….). Pour info, tout est faux, c’est une api pour s'entraîner.

Algorithmie - AJAX



Autre exemple, le site “vitemadose”, utilise les api des plateformes de prise de rendez-vous et les affiche directement dans son site

Algorithmie - Promesses et callback

Dans la vie de tous les jours, on dit que deux actions sont synchrones lorsqu'elles se déroulent en même temps ou de manière synchronisée. Au contraire, deux opérations sont asynchrones si elles ne se déroulent pas en même temps ou ne sont pas synchronisées.

En informatique, on dit que deux opérations sont synchrones lorsque la seconde attend que la première ait fini son travail pour démarrer. En bref : le début de l'opération suivante dépend de la complétude de l'opération précédente.

Au contraire, deux opérations sont qualifiées d'asynchrones en informatique lorsqu'elles sont indépendantes c'est-à-dire lorsque la deuxième opération n'a pas besoin d'attendre que la première se termine pour démarrer.

Par défaut, le JavaScript est un langage synchrone, bloquant et qui ne s'exécute que sur un seul thread. Cela signifie que :

- Les différentes opérations vont s'exécuter les unes à la suite des autres (elles sont synchrones) ;

- Chaque nouvelle opération doit attendre que la précédente ait terminé pour démarrer (l'opération précédente est « bloquante ») ;

- Le JavaScript ne peut exécuter qu'une instruction à la fois (il s'exécute sur un thread, c'est-à-dire un « fil » ou une « tache » ou un « processus » unique).

Algorithmie - Promesses et callback

```
console.log('je suis le premier')


fetch('https://vitemadose.gitlab.io/vitemadose/14.json')
.then((data)=>{
    return data.json();
})
.then((data)=>{
    console.log('je suis le deuxième')
})


console.log('je suis le troisième')
console.log('je suis le dernier')
```



On voit que le deuxième console.log est exécuté en dernier car il est dans une “promise” (promesse) et donc n’est pas dans le fil principal (main thread) mais dans un thread secondaire.

Algorithmie - Promesses et callback

```
const getPosts = async function(){

    let persons = await fetch('https://jsonplaceholder.typicode.com/users');
    persons = await persons.json();
    let comments = await fetch('https://jsonplaceholder.typicode.com/comments?userId='+persons[0]);
    comments = await comments.json();
    return comments;

}

getPosts().then(function(comments){
    console.log(comments);
})
```

Maintenant, il y a une façon encore plus propre d'écrire des "promises". Grace aux await et async